

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
«Гомельский государственный технический университет
имени П.О. Сухого»

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

направление специальности 6-05-0611-01 Информационные системы и
технологии (в проектировании и разработке программного обеспечения
информационных систем)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
по дисциплине «Распределённые информационные системы»

на тему: Разработка распределённой системы телеметрии
и адаптивного освещения на базе *ESP32*

Исполнитель: студент группы ИТП-31
Сеноженский В.В.

Руководитель: доцент
Комраков В. В.

Дата проверки: _____

Дата допуска к защите: _____

Дата защиты: _____

Оценка работы: _____

Подписи членов комиссии
по защите курсового проекта: _____

Гомель 2026

СОДЕРЖАНИЕ

Введение	4
1 Теоретический и программно-аппаратный обзор предметной области.....	5
1.1 Сравнительная характеристика датчиков движения и присутствия..	5
1.2 Физические основы радиолокации и распространения СВЧ-излучения	6
1.3 Сравнительный анализ микроконтроллерных платформ для телеметрии	8
1.4 Архитектура программного обеспечения	9
2 Архитектура аппаратно-программного комплекса	12
2.1 Назначение и требования к аппаратно-программному комплексу..	12
2.2 Аппаратная архитектура и схема лабораторного полигона	13
2.3 Программная архитектура	17
2.4 Среда разработки и алгоритмическое обеспечение микроконтроллера	17
2.5 Приложение для детекции объектов	18
2.6 Основные результаты проектирования	19
3 Практическая реализация и характеристики программно-аппаратного комплекса.....	21
3.1 Структура программного обеспечения и диаграммы взаимодействия	21
3.2 Логическая и физическая структура базы данных	22
3.3 Графический интерфейс пользователя и визуализация процессов .	23
3.4 Конфигурация аппаратных зон чувствительности СВЧ-радара	26
3.5 Результаты экспериментов и оценка диэлектрического затухания СВЧ-сигнала	28
Заключение	31
Список использованных источников	31
Приложение А Диаграммы эксплуатируемого ПО	33
Приложение Б Листинг программы	33
Приложение В Руководство системного программиста	49
Приложение Г Руководство пользователя	50
Приложение Д Руководство программиста	51

ВВЕДЕНИЕ

Современные системы телеметрии и интернета вещей (*IoT*) стремительно развиваются, требуя применения все более точных и надежных сенсорных технологий. В последние годы наблюдается устойчивая тенденция перехода от традиционных инфракрасных и ультразвуковых датчиков к радиолокационным модулям, работающим в диапазоне сверхвысоких частот. Использование *FMCW*-радаров *K*-диапазона открывает новые возможности для создания высокоточных систем мониторинга, умного дома и неразрушающего контроля материалов.

Актуальность разработки программно-аппаратных комплексов на базе СВЧ-радаров обусловлена необходимостью точного бесконтактного детектирования не только движущихся, но и статичных объектов, а также микро-движений. Кроме того, интеграция таких датчиков с современными микроконтроллерами, периферийными устройствами и высокоуровневым программным обеспечением для анализа данных позволяет исследовать физические свойства материалов, такие как радиопрозрачность и диэлектрическая проницаемость, без использования дорогостоящего лабораторного оборудования.

Целью данного курсового проекта является разработка распределенного аппаратно-программного комплекса телеметрии на базе микроконтроллера *ESP32* и радарного датчика *HLK-LD2410C* (24 ГГц) для сбора, фильтрации, визуализации данных и практического исследования затухания СВЧ-радиоволн при прохождении через преграды с различной диэлектрической проницаемостью.

Для достижения поставленной цели необходимо провести анализ предметной области, изучить физические принципы работы *FMCW*-радаров и особенности распространения микроволн. Спроектировать и собрать аппаратную часть лабораторного полигона на базе *ESP32* с соблюдением требований электробезопасности, корректным подключением адресной светодиодной ленты *WS2812B* и устранением паразитных наводок. Разработать микропрограммное обеспечение на языке *C++* для микроконтроллера, реализующее непрерывный опрос датчика, фильтрацию шумовых артефактов и управление визуальной индикацией. Разработать кроссплатформенное десктопное приложение на языке *Python* с использованием библиотеки *PyQt5* для приема телеметрии, ведения базы данных, математической обработки и построения динамических графиков в реальном времени. Разработать методику и провести серию натурных экспериментов по оценке радиопрозрачности различных диэлектриков (воздух, пластик, вода).

Объектом исследования является процесс распространения и затухания электромагнитных волн сверхвысокой частоты в различных средах.

Предметом исследования выступает программно-аппаратная реализация системы телеметрии для захвата, обработки и визуализации радиолокационных данных.

1 ТЕОРЕТИЧЕСКИЙ И ПРОГРАММНО-АППАРАТНЫЙ ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительная характеристика датчиков движения и присутствия

Выбор аппаратной платформы и сенсорной базы является фундаментальным этапом проектирования любой системы телеметрии. От физического принципа работы датчика зависят точность измерений, устойчивость системы к внешним помехам и возможность работы в сложных условиях эксплуатации. В рамках данного курсового проекта рассматривается задача бесконтактного обнаружения и локализации объектов сквозь радиопрозрачные преграды.

Исторически наиболее распространенным решением для обнаружения движения являются пассивные инфракрасные датчики, такие как популярный модуль *HC-SR501* [1]. Их принцип действия основан на регистрации изменений теплового излучения в рабочей зоне. Несмотря на низкую стоимость и низкое энергопотребление, *PIR*-датчики обладают рядом критических недостатков, делающих их непригодными для решения задач данного проекта. Во-первых, они способны фиксировать только перемещение объектов с выраженным тепловым контрастом относительно фона, неподвижный человек или объект со временем сливается с фоном и исчезает из поля зрения датчика. Во-вторых, *PIR*-сенсоры абсолютно не способны работать сквозь преграды, так как эти материалы поглощают или отражают инфракрасные лучи.

Альтернативным решением являются ультразвуковые дальномеры, использующие принцип эхолокации акустических волн. Ультразвуковые датчики позволяют с высокой точностью измерять расстояние до объекта путем вычисления времени задержки отраженного сигнала. Однако акустические волны обладают низкой проникающей способностью и подвержены сильному рассеиванию на мягких поверхностях. Кроме того, они не могут проникать сквозь твердые диэлектрики, так как акустическая волна отражается от первой же границы раздела сред с разной акустической плотностью.

Оптические датчики и системы компьютерного зрения на базе видеокамер обеспечивают высочайшую точность и информативность, однако требуют прямой видимости объекта, значительных вычислительных мощностей для обработки видеопотока и не способны функционировать в условиях отсутствия освещения или за оптически непрозрачными преградами.

В противовес вышеописанным технологиям, радиолокационные датчики сверхвысокой частоты используют электромагнитные волны для зондирования пространства. Радиоволны *K*-диапазона обладают уникальной комбинацией физических свойств: длиной волны порядка 12,5 мм, что обеспечивает высокую разрешающую способность, и способностью свободно проникать сквозь большинство неметаллических диэлектриков.

Для реализации проекта был выбран модуль *HLK-LD2410C* – компактный СВЧ-радар, использующий принцип непрерывного частотно-модулированного излучения. В отличие от простых доплеровских радаров, способных измерять

только скорость движущегося объекта, технология *FMCW* позволяет одновременно вычислять и скорость, и точную дистанцию до цели путем анализа разности частот излученного и принятого сигналов.

Ключевым преимуществом модуля *LD2410C* является наличие встроенного цифрового сигнального процессора (*DSP*) со сложными алгоритмами фильтрации и адаптивного вычитания фона. Данный процессор способен анализировать не только макро-движения, но и микро-сдвиги фазы отраженной волны, вызванные, например, дыханием статично сидящего человека. Это позволяет радару отличать живые объекты от неодушевленных элементов интерьера, а также предоставляет возможность аппаратного зонирования пространства с индивидуальной настройкой чувствительности для каждой зоны.

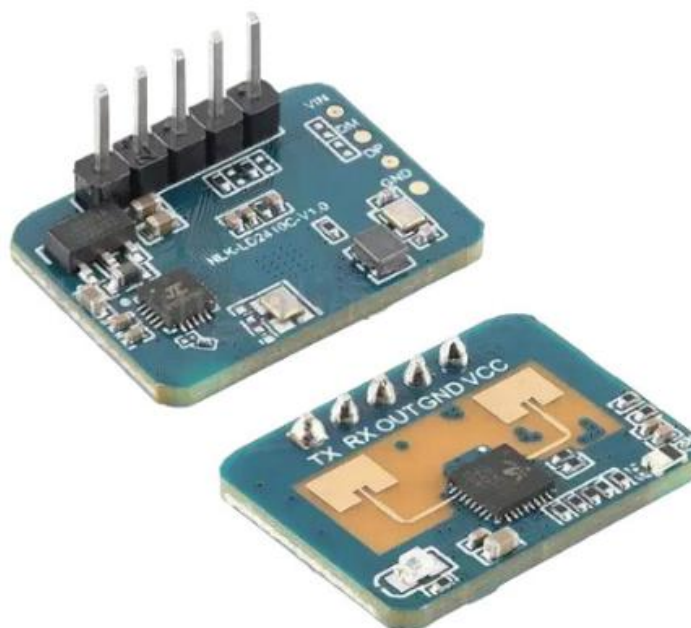


Рисунок 1.1 – Микроволновой датчик *LD2410C*

Таким образом, выбор СВЧ-радара *HLK-LD2410C* обоснован его способностью решать комплексную задачу проекта: измерять уровень отраженной энергии и дистанцию до цели сквозь радиопрозрачные преграды, игнорируя при этом тепловые и акустические помехи [2].

1.2 Физические основы радиолокации и распространения СВЧ-излучения

Основой функционирования разработанного аппаратно-программного комплекса является принцип радиолокации – процесса обнаружения, распознавания и определения местоположения объектов с помощью электромагнитных волн радиодиапазона. В проекте используется радиолокатор,

генерирующий электромагнитные колебания на частоте 24 ГГц, что соответствует длине волны $\lambda \approx 12,5$ мм в вакууме.

Ключевой характеристикой СВЧ-радара является способность излучаемых им электромагнитных волн взаимодействовать с различными материалами окружающей среды. Характер этого взаимодействия (отражение, прохождение, преломление или поглощение) фундаментально зависит от электрофизических свойств материала, главной из которых выступает относительная диэлектрическая проницаемость (ϵ).

Диэлектрическая проницаемость ϵ – это безразмерная физическая величина, характеризующая степень поляризации диэлектрика под воздействием внешнего электрического поля. В контексте высокочастотной радиолокации данный параметр определяет, какая доля энергии электромагнитной волны отразится от границы раздела сред, а какая доля проникнет внутрь материала и, возможно, будет им поглощена [3].

Материалы с низкой диэлектрической проницаемостью являются радиопрозрачными для частоты 24 ГГц. При падении СВЧ-луча на преграду из такого материала, коэффициент отражения на границе сред минимален. Основная часть электромагнитной энергии проходит сквозь преграду практически без ослабления амплитуды. В экспериментальной части данного проекта пустая пластиковая тара моделирует радиопрозрачную преграду. Незначительные потери энергии отраженного от мишени сигнала в этом случае обусловлены не поглощением, а эффектом френелевского отражения и рассеиванием луча из-за геометрии цилиндрической поверхности.

Принципиально иная физическая картина наблюдается при взаимодействии СВЧ-излучения с материалами, обладающими высокой диэлектрической проницаемостью. Ярчайшим примером такого вещества является вода ($\epsilon \approx 80$ при комнатной температуре). Высокое значение ϵ воды обусловлено тем, что молекула H_2O представляет собой ярко выраженный электрический диполь.

Когда электромагнитная волна частотой 24 ГГц проникает в толщу воды, она заставляет дипольные молекулы воды совершать вынужденные вращательные колебания. На преодоление сил межмолекулярного трения затрачивается колоссальное количество энергии волны, которая необратимо преобразуется в тепловую энергию. Этот процесс называется диэлектрическим поглощением [4].

Кроме того, из-за гигантского скачка диэлектрической проницаемости на границе сред возникает сильнейшее отражение падающей волны. Коэффициент отражения по мощности от водной поверхности может достигать 50-60%. Оставшаяся часть волны, проникшая внутрь, испытывает экспоненциальное затухание.

Важнейшим параметром, характеризующим способность материала поглощать электромагнитное излучение, является глубина скин-слоя – расстояние, на котором амплитуда поля убывает в $e \approx 2,71$ раз. Для воды на частоте 24 ГГц глубина скин-слоя составляет всего около 1-2 миллиметров. Это означает, что преграда из воды толщиной в несколько сантиметров действует как

абсолютно непроницаемый СВЧ-щит, полностью блокируя прохождение сигнала к мишени, находящейся за ней.

Металлы обладают бесконечно большой диэлектрической проницаемостью и высокой проводимостью. Электромагнитная волна не проникает внутрь идеального проводника, вместо этого она индуцирует на его поверхности поверхностные токи которые переизлучают волну обратно. Таким образом, металл выступает идеальным СВЧ-зеркалом, обеспечивая максимальный уровень энергии отраженного сигнала, фиксируемого приемной антенной радара.

1.3 Сравнительный анализ микроконтроллерных платформ для телеметрии

Для управления радарным модулем, первичной обработки радиолокационных данных, реализации алгоритмов фильтрации и организации канала связи с персональным компьютером требуется использование встраиваемой вычислительной платформы. Выбор аппаратной базы напрямую влияет на производительность, стабильность работы ПАК и возможность его дальнейшего масштабирования.

В современной практике прототипирования систем телеметрии наиболее широкое распространение получили платформы семейств *Arduino* (на базе AVR-архитектуры) и *ESP* (на базе *Xtensa*-архитектуры).

Классическая платформа *Arduino UNO*, построенная на базе 8-битного микроконтроллера *ATmega328P* с тактовой частотой 16 МГц и объемом оперативной памяти всего 2 КБ, является отличным решением для простых задач автоматизации. Однако для реализации данного проекта ее ресурсов недостаточно по ряду причин [5].

Во-первых, модуль *HLK-LD2410C* генерирует непрерывный поток данных по интерфейсу *UART* на высокой скорости. Обработка такого потока требует аппаратной поддержки высокоскоростных последовательных портов и достаточного объема буферной памяти. Микроконтроллер *ATmega328P* имеет лишь один аппаратный *UART*, который обычно занят связью с ПК, а программная эмуляция *SoftwareSerial* на скорости 256000 бод невозможна из-за высоких накладных расходов на процессорное время, что неизбежно приведет к потере пакетов телеметрии.

Во-вторых, низкая вычислительная мощность *Arduino UNO* не позволяет реализовывать сложные неблокирующие алгоритмы и математическую фильтрацию сигналов в реальном времени, параллельно управляя периферийными устройствами.

Альтернативным решением класса одноплатных компьютеров является платформа *Raspberry Pi* [6]. Данная архитектура обладает избыточной для решаемой задачи вычислительной мощностью и функционирует под управлением полноценной ОС *Linux*. Использование *Raspberry Pi* в качестве контроллера сбора телеметрии нецелесообразно с экономической точки зрения,

требует сложной организации питания и усложняет архитектуру из-за необходимости обслуживания ОС реального времени.

Оптимальным выбором для реализации аппаратной части ПАК стал микроконтроллерный модуль *ESP32* (в модификации *ESP32 DevKitC V4*). Данный модуль построен на базе высокопроизводительного двухъядерного 32-битного процессора *Xtensa LX6*, работающего на частоте до 240 МГц.

Платформа *ESP32* имеет следующие технические преимущества:

- наличие трех аппаратных интерфейсов *UART* – это позволяет выделить один независимый высокоскоростной порт (*UART2*) исключительно для обмена данными с радарным датчиком на скорости 256000 бод, оставив основной порт (*UART0*) для трансляции телеметрии в десктопное приложение на ПК (на скорости 115200 бод), исключая коллизии и переполнение буферов;

- высокий объем оперативной памяти обеспечивает возможность хранения массивов данных для усреднения, надежной буферизации *UART*-пакетов и стабильной работы библиотек управления адресной светодиодной индикацией;

- высокая производительность, частота 240 МГц гарантирует мгновенное выполнение алгоритмов парсинга датаграмм радара, фильтрации дистанции и соблюдение жестких микросекундных таймингов при отправке данных на ленту *WS2812B*;

- задел на модернизацию (*IoT*) – это наличие встроенных модулей *Wi-Fi* и *Bluetooth* позволяет в будущем легко преобразовать проводной лабораторный ПАК в полностью автономный беспроводной узел системы умный дом с передачей телеметрии на удаленный *MQTT*-брокер.

1.4 Архитектура программного обеспечения

Разработка распределенного программно-аппаратного комплекса требует тщательного выбора архитектурного паттерна и стека технологий, способных обеспечить не только сбор сырых данных с аппаратной периферии, но и их надежную высокоуровневую обработку, хранение и визуализацию в реальном времени.

Существующие архитектурные решения для систем мониторинга и сбора данных можно разделить на две крупные категории: веб-ориентированные и локальные десктопные приложения.

Веб-ориентированная архитектура предполагает наличие выделенного веб-сервера, базы данных (*СУБД MySQL, PostgreSQL*) и тонкого клиента в виде браузера. Подобный подход является стандартом де-факто для построения многопользовательских систем управления предприятием (*ERP*), систем технического обслуживания или глобальных *IoT*-платформ, где требуется доступ из любой точки мира через сеть Интернет и жесткое разграничение ролей пользователей [7].

Однако для решения задач радиолокационного комплекса, требующего работы с аппаратными интерфейсами ПК (*COM*-портами) на высоких скоростях

и отрисовки динамических графиков с частотой обновления до нескольких десятков герц, веб-архитектура обладает рядом критических недостатков:

- проблема прямого доступа к оборудованию: браузерные технологии в целях безопасности изолированы от аппаратной части ПК. Прямое чтение данных из *USB/UART*-портов невозможно без создания промежуточного локального сервиса-прослойки, что неоправданно усложняет архитектуру;

- высокие задержки и накладные расходы: передача потоковых телеметрических данных по протоколам *HTTP/REST* или даже *WebSocket* сопровождается значительным объемом служебного трафика, что может привести к потере пакетов и рассинхронизации при пиковых нагрузках;

- избыточность инфраструктуры: развертывание полноценного веб-сервера и реляционной СУБД для локального сбора экспериментальных данных на одном лабораторном стенде является архитектурным оверинжинирингом, усложняющим запуск и отладку комплекса.

Ввиду вышеизложенного, для реализации программной части ПАК была выбрана архитектура локального десктопного приложения на базе высокоуровневого языка программирования *Python*.

Python является наиболее эффективным в сфере *Data Science*, машинного обучения и лабораторной автоматизации. Богатая экосистема открытых библиотек позволяет в кратчайшие сроки реализовать как низкоуровневое взаимодействие с оборудованием (библиотека *pyserial*), так и сложную математическую обработку массивов данных (библиотека *pandas*).

В качестве механизма хранения экспериментальных данных был выбран формат *CSV (Comma-Separated Values)*. В отличие от тяжеловесных реляционных СУБД (*MySQL*), требующих установки отдельного сервера баз данных, *CSV*-файлы представляют собой легковесные плоские таблицы, хранящиеся локально на жестком диске. Данный подход обеспечивает мгновенную запись новых транзакций, легкость резервного копирования и 100% совместимость с любыми внешними аналитическими инструментами (*MS Excel*, *MATLAB*, *R*) без необходимости написания сложных *SQL*-запросов для выгрузки результатов. Использование библиотеки *pandas* позволяет загружать весь *CSV*-файл в оперативную память и выполнять агрегирующие вычисления (группировку по материалам, расчет средних значений энергии и дистанции) за доли секунды.

Для реализации пользовательского графического интерфейса (*GUI*) используется библиотека *PyQt5*. В отличие от встроенных, но устаревших решений (например, *tkinter*), *PyQt5* представляет собой *Python*-обертку над мощным *C++* фреймворком *Qt*, являющимся индустриальным стандартом для создания кроссплатформенного инженерного и научного программного обеспечения[8].

Использование *PyQt5* обеспечивает следующие преимущества:

- высокая производительность отрисовки (*Hardware Acceleration*) позволяет создавать плавные анимации физических процессов с частотой 30-60 *FPS* без блокировки основного потока приложения;

- интеграция с математическими плоттерами, библиотека *matplotlib*, используемая для построения профессиональных аналитических графиков

затухания СВЧ-излучения, нативно встраивается в виджеты *PyQt5* (через класс *FigureCanvasQTAgg*), образуя единый монолитный дашборд;

– богатый набор компонентов, наличие продвинутых элементов управления.

Выбранный стек технологий – *C++* для микроконтроллера *ESP32*, *Python* + *PyQt5* + *Pandas* для ПК. Аппаратный уровень обеспечивает жесткий реал-тайм сбор и фильтрацию СВЧ-телеметрии.

Такое четкое разделение вычислительной нагрузки между микроконтроллером и персональным компьютером позволяет избежать потери данных при пиковых частотах обновления радарного датчика. Программная часть на стороне ПК, освобожденная от необходимости низкоуровневого контроля аппаратных таймингов, полностью задействует свои ресурсы для бесперебойной отрисовки пользовательского интерфейса, выполнения математических вычислений и работы с файловой системой. В результате формируется надежная и масштабируемая архитектура, которая не только полностью отвечает всем заявленным требованиям к лабораторному измерительному стенду, но и оставляет существенный задел для будущего внедрения более сложных алгоритмов аналитики без необходимости переписывать базовую логику обмена данными.

2 АРХИТЕКТУРА АППАРАТНО-ПРОГРАММНОГО КОМПЛЕКСА

2.1 Назначение и требования к аппаратно-программному комплексу

Проектирование аппаратно-программного комплекса для исследования характеристик СВЧ-излучения продиктовано потребностью в создании защищенного от радиопомех измерительного инструмента. Базовый радарный модуль *HLK-LD2410C* изначально ориентирован на применение в экосистемах интернета вещей в роли детектора присутствия биологических объектов. Встроенные алгоритмы цифровой обработки сигналов данного сенсора настроены на подавление статических отражений от неживых объектов, расцениваемых как фоновый шум. Адаптация модуля под задачи высокоточного измерительного прибора требует разработки специализированной программной архитектуры, способной нивелировать встроенные аппаратные фильтры.

Основная цель проектирования заключается в построении отказоустойчивой системы с поддержкой бесконтактной инициализации измерений. Наличие биологического объекта в зоне распространения СВЧ-луча провоцирует критические искажения радиоэфира, возникновение многолучевого распространения и перенасыщение приемного тракта радара. Архитектура комплекса обязана предоставлять возможность дистанционного запуска цикла сканирования с последующим освобождением рабочей зоны, что гарантирует получение математически очищенных данных об энергии отражения строго от исследуемой мишени [9].

Для успешной реализации комплекса сформирован перечень строгих технических и функциональных требований, разделенный на три базовых уровня.

Требования к аппаратной части и топологии полигона:

- изолированность антенны: измерительный стенд должен исключать формирование паразитных переотражений в слепой зоне. Требуется установка радарного модуля на границе монтажной поверхности, исключающая попадание опорной плоскости в СВЧ-конус;

- геометрия измерений: система обязана поддерживать строго линейное распространение сигнала. Эталонный СВЧ-отражатель фиксируется на дистанции 100 см. Пространство для размещения исследуемых диэлектрических преград локализуется на отметке 50 см, строго по центру оси излучения;

- электробезопасность: аппаратная обвязка на базе микроконтроллера *ESP32* и адресной светодиодной ленты *WS2812B* должна содержать защиту от паразитного обратного питания. Требуется полное исключение протекания токов по информационным шинам при обесточенной силовой линии посредством внедрения регламента безопасного запуска комплекса.

Требования к микропрограммному обеспечению:

- высокоскоростной опрос: микроконтроллер *ESP32* обязан выполнять непрерывное чтение *UART*-буфера на скорости 256000 бод, предотвращая переполнение оперативной памяти и потерю телеметрических пакетов от радара;

– событийная архитектура: прошивка должна содержать логику гибридного триггера – запуск алгоритма измерения допускается исключительно при фиксации ярко выраженного движения с пороговым значением энергии более 40 единиц;

– асинхронные задержки: для обеспечения временного интервала на освобождение зоны измерения алгоритму предписывается использование неблокирующих функций тайминга с параллельной фоновой очисткой входящего буфера данных;

– пространственная фильтрация: микропрограмма должна на аппаратном уровне отсекал любые пакеты данных о целях, находящихся на дистанции ближе 75 см, транслируя на верхний уровень исключительно параметры эталонной мишени.

Требования к программному обеспечению верхнего уровня:

– асинхронный интерфейс: десктопное приложение должно базироваться на фреймворке *PyQt5*, обладать современным графическим интерфейсом и сохранять отзывчивость во время ожидания пакетов по *SOM*-порту;

– агрегация данных: программа обязана формировать пакет из 4-6 последовательных кадров радара, вычисляя среднее арифметическое значение дистанции и энергии для минимизации статистического джиттера;

– управление хранилищем: приложение должно самостоятельно сохранять результаты измерений и параметры преград в плоскую файловую базу данных формата *CSV*. Требуется наличие встроенных инструментов для управления записями непосредственно из графического интерфейса;

– аналитика и визуализация: обязательным требованием является интеграция модуля отрисовки аналитических гистограмм затухания сигналов, а также подсистемы генерации анимации распространения СВЧ-волн в реальном времени, зависящей от электрофизических свойств выбранной преграды[10].

Реализация заявленных требований предполагает создание отказоустойчивой распределенной архитектуры. Аппаратный уровень берет на себя жесткий реал-тайм сбор и первичную фильтрацию СВЧ-телеметрии, программный уровень отвечает за математическую аналитику и визуальное представление скрытых физических процессов.

2.2 Аппаратная архитектура и схема лабораторного полигона

Аппаратная часть комплекса формируется из трех ключевых функциональных узлов: управляющего микроконтроллера, модуля СВЧ-радар и модуля визуальной индикации.

Монтаж компонентов проектируется на базе беспаячной макетной платы с применением внешнего модуля питания *YwRobot MB102*. Данный модуль выполняет понижение напряжения от внешнего источника питания. Внедрение мощного внешнего источника питания продиктовано высоким пиковым потреблением тока адресной светодиодной. Встроенный линейный регулятор напряжения на плате *ESP32* не рассчитан на подобные токи, и попытка запитать

ленту напрямую от микроконтроллера неизбежно приведет к термическому разрушению компонентов [11].

Структурная схема электрических соединений разрабатываемого аппаратно-программного комплекса представлена на рисунке 2.1.

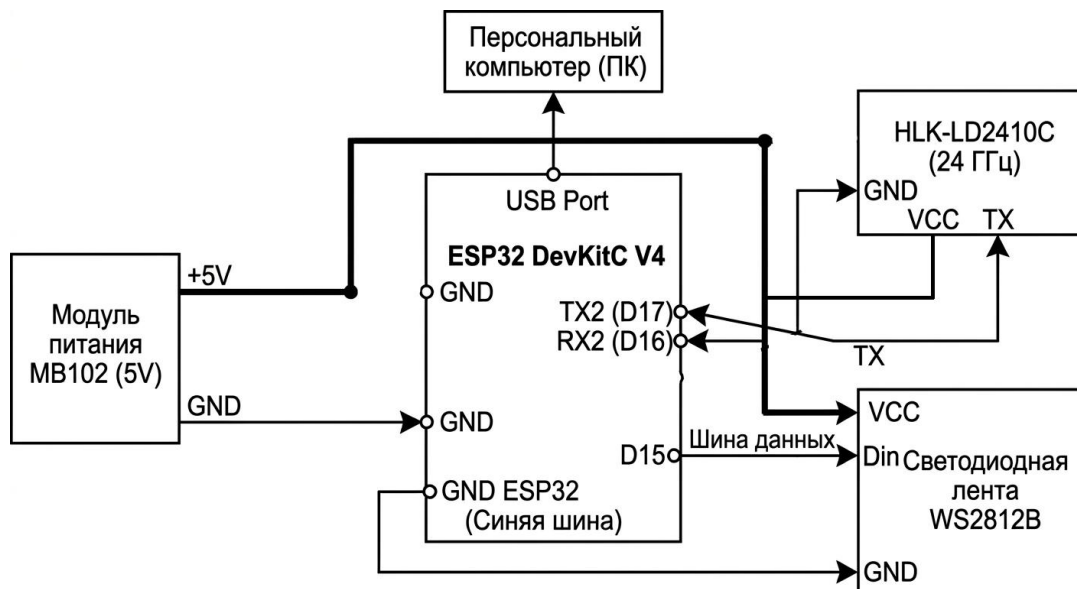


Рисунок 2.1 – Структурная электрическая схема аппаратной части ПАК

При проектировании схемы соединений вводится общая шина заземления (*GND*), объединяющая минусовую линию модуля *MB102*, пин *GND* микроконтроллера *ESP32*, вывод *GND* радара и вывод *GND* светодиодной ленты. Силовая шина *+5V* обеспечивает независимое питание радара и светодиодной ленты непосредственно от модуля *MB102*, не пересекаясь с цепями питания микроконтроллера (*ESP32* получает изолированное питание через собственный *USB*-порт от персонального компьютера) [12].

Информационный обмен между компонентами строится по следующей схеме:

- управление светодиодной лентой (шина данных) осуществляется через цифровой вывод *D15* микроконтроллера *ESP32*;
- прием и передача телеметрии радара организованы по аппаратному интерфейсу *UART2* микроконтроллера. Вывод *TX* радара подключается к выводу *RX2* (*D16*) *ESP32*, а вывод *RX* радара – к выводу *TX2* (*D17*) *ESP32*, реализуя классическое перекрестное соединение.

В целях защиты цифрового порта *D15* микроконтроллера от токовых выбросов, возникающих при резком изменении яркости адресной светодиодной ленты, в разрыв информационного провода (шины данных) установлен токоограничивающий резистор номиналом 330 Ом. Физическая реализация аппаратного узла на базе *ESP32 DevKitC V4* представлена на рисунке 2.2.

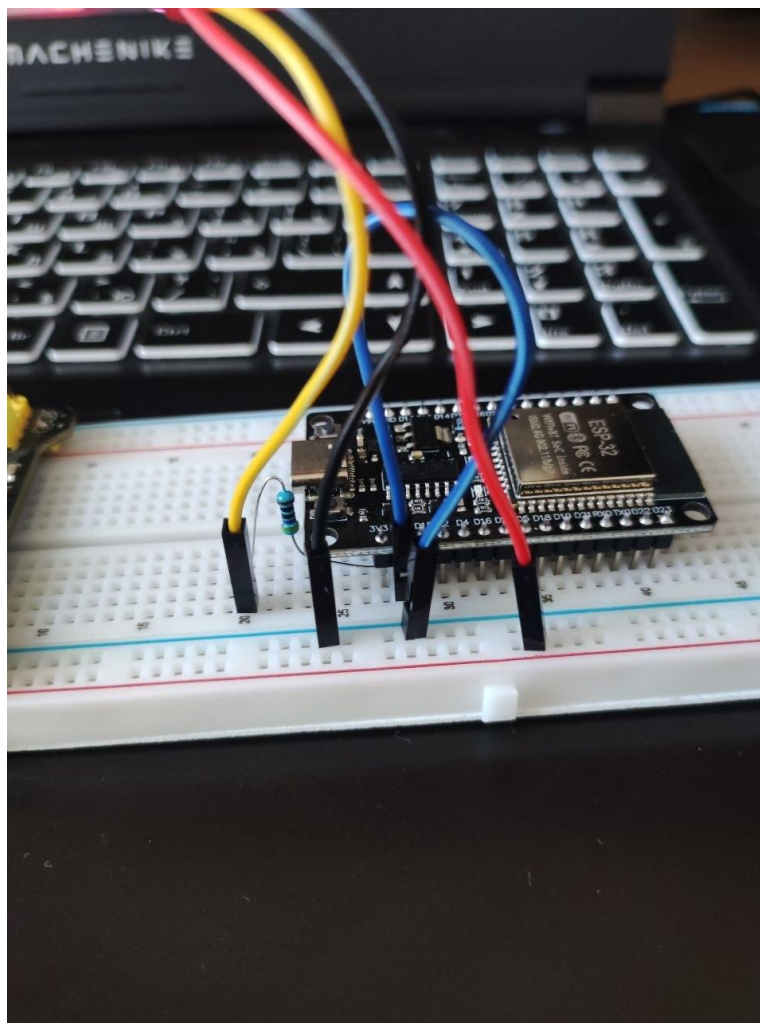


Рисунок 2.2 – Внешний вид аппаратной сборки управляющего узла измерительного комплекса

Критическим аспектом проектирования аппаратной архитектуры выступает предотвращение эффекта паразитного обратного питания. Анализ схемотехники применяемых компонентов показывает, что при обесточивании силовой линии +5V (отключение внешнего блока питания) и сохранении *USB*-подключения *ESP32* к ПК, напряжение 3,3 В с логических выводов микроконтроллера (*D15*, *D16*, *D17*) начинает протекать в обратном направлении через внутренние защитные диоды периферийных устройств (радара и ленты). Данный процесс вызывает нештатное свечение светодиодов и перегрузку информационных портов *ESP32*, что ведет к выходу микроконтроллера из строя [13].

Для исключения аппаратных повреждений в регламент эксплуатации комплекса закладывается строгая последовательность подачи напряжений. Инициализация системы допускается исключительно путем первоначального включения силовой линии +5V с последующим подключением *USB*-кабеля для обеспечения логического обмена с ПК. Процедура прошивки микроконтроллера требует физического разрыва соединений информационных шин радара и светодиодной ленты. В качестве дополнительной меры стабилизации питания,

предотвращающей перезагрузку микроконтроллера при резких скачках тока, в проект закладывается установка электролитического конденсатора емкостью 1000 мкФ параллельно силовым шинам от модуля *MB102*.

Пространственная конфигурация измерительного полигона проектируется с учетом диаграммы направленности антенны радара *HLK-LD2410C*. Радар обладает конусообразной зоной покрытия с углом раскрытия луча $\approx 60^\circ$ в горизонтальной и вертикальной плоскостях.

Монтаж радарного модуля предусматривает его размещение на границе плоской поверхности. Излучающая антенна выносится за край поверхности, что полностью исключает отражение СВЧ-сигнала от опорной плоскости в ближней слепой зоне и предотвращает ослепление приемного тракта.

Геометрия измерительного стенда предполагает линейное размещение объектов. Эталонная мишень, представляющая собой цилиндрический объект со 100% СВЧ-отражением (металлическая банка или стеклянная колба, обернутая фольгой), фиксируется на дистанции 100 см от излучателя. На отметке 50 см, строго на линии прямого визирования, организуется зона для установки диэлектрических преград. Такая конфигурация гарантирует, что тестируемая преграда перекрывает центральную ось СВЧ-конуса, обеспечивая максимальную достоверность измеряемых коэффициентов затухания.

2.3 Программная архитектура

Программное обеспечение разработанного программно-аппаратного комплекса имеет распределенную двухуровневую структуру, разделенную на уровень встроенного программного обеспечения микроконтроллера *ESP32* и прикладной уровень десктопного приложения на базе языка *Python*. Такое разделение позволяет оптимально распределить вычислительную нагрузку: задачи реального времени, первичного накопления и аппаратной фильтрации сигналов выполняются на энергоэффективном микроконтроллере, в то время как ресурсоемкие процессы хранения, статистической обработки, аналитики и визуализации данных перенесены на персональный компьютер.

Для построения архитектуры прикладного программного обеспечения верхнего уровня использован классический шаблон проектирования *MVC*. Применение данного паттерна обеспечивает слабую связанность компонентов, упрощает масштабирование системы и модификацию алгоритмов обработки данных без изменения графического интерфейса.

Компоненты шаблона *MVC* в разработанном программном комплексе распределены следующим образом:

- *model* представлена классом *DataEngine*. Модель полностью изолирована от графического интерфейса и отвечает за непосредственное взаимодействие с хранилищем данных в формате *CSV*. С использованием библиотеки *pandas* модель реализует чтение, валидацию и фильтрацию данных, а также выполняет расчет агрегированных показателей.

- *view* включает графический интерфейс пользователя, построенный на базе библиотеки *PyQt5*, интерактивные аналитические графики *matplotlib*, а

также специализированный графический движок реального времени *RadarAnimation*. Класс *RadarAnimation* наследуется от *QWidget* и с помощью низкоуровневого класса *QPainter* выполняет отрисовку физических процессов прохождения СВЧ-волн через преграду на основе полученных от модели данных об уровне энергии.

– *controller* реализован внутри основного класса *RadarApp*. Он координирует взаимодействие между моделью и представлением, обрабатывает сигналы пользовательского интерфейса и управляет асинхронным приемом телеметрии из последовательного порта.

Схема программной архитектуры приведена на рисунке 2.3.

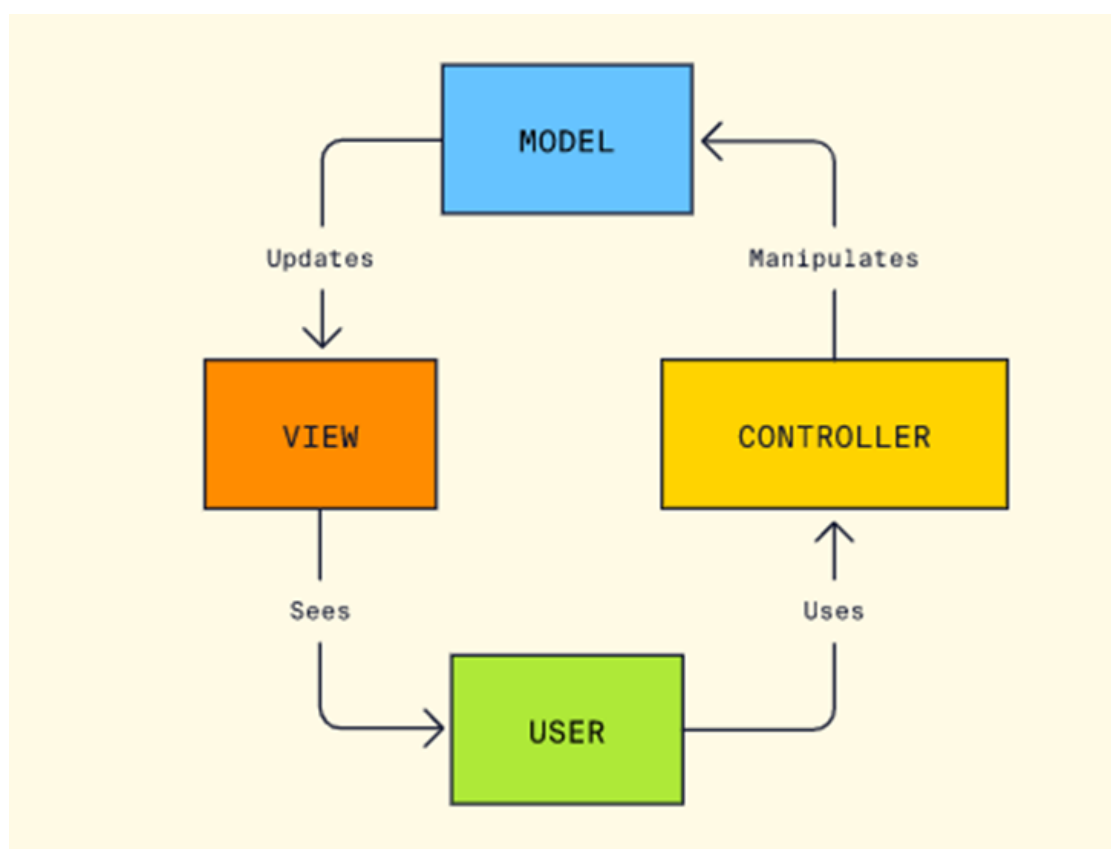


Рисунок 2.3 – Схема программной архитектуры

2.4 Среда разработки и алгоритмическое обеспечение микроконтроллера

Для реализации логики управления аппаратной частью ПАК требуется разработка микропрограммного обеспечения микроконтроллера *ESP32*. В качестве среды разработки рассматривались два основных решения: официальный фреймворк *Espressif IoT Development Framework* и платформа *Arduino IDE*.

Использование *ESP-IDF* предоставляет максимальный контроль над аппаратными ресурсами и операционной системой реального времени, однако требует значительных временных затрат на разработку и отладку

низкоуровневых драйверов. Платформа *Arduino IDE* является более эффективным решением. Главным преимуществом данной среды для проектируемого комплекса является наличие обширной экосистемы готовых и протестированных библиотек. В частности, интеграция библиотеки *Adafruit_NeoPixel* позволяет реализовать жесткие микросекундные тайминги, необходимые для управления адресной светодиодной лентой *WS2812B*, без написания сложного ассемблерного кода. Наличие встроенного класса *HardwareSerial* значительно упрощает конфигурацию интерфейса *UART2* для высокоскоростного (256000 бод) обмена данными с радаром *HLK-LD2410C*.

Проектирование алгоритма работы микроконтроллера продиктовано необходимостью обхода аппаратных ограничений цифрового сигнального процессора радара. Встроенная логика датчика классифицирует неподвижные объекты как фоновый шум, удаляя их из буфера. Для решения этой проблемы алгоритм микроконтроллера проектируется на базе концепции конечного автомата с применением событийно-ориентированной архитектуры.

При проектировании автомата состояний необходимо решить критическую проблему переполнения буфера. Использование стандартных функций блокирующей задержки для организации паузы недопустимо. При блокировке процессора на 3 секунды непрерывный поток датаграмм от радара вызовет переполнение аппаратного *UART*-буфера *ESP32*, что приведет к потере синхронизации и искажению пакетов данных.

Ожидание реализуется через системный таймер *millis()*. Внутри цикла задержки происходит непрерывный вызов функции чтения данных с радара, что обеспечивает фоновую очистку приемного буфера и гарантирует обработку только актуальных пакетов после завершения паузы [14].

Для обеспечения чистоты эксперимента программный алгоритм должен содержать механизм пространственной фильтрации. Применение логического условия (проверка дистанции $dist \geq 75$) непосредственно в коде *C++* позволяет аппаратно отсекают любые радиочастотные отклики ближней зоны. На верхний уровень транслируются исключительно параметры объектов, находящихся в зоне расположения эталонной мишени, что минимизирует нагрузку на интерфейс *USB* и исключает передачу ложных данных.

2.5 Приложение для детекции объектов

Для приема, обработки, хранения и визуализации телеметрических данных требуется разработка программного обеспечения верхнего уровня. В качестве языка программирования *Python* является наиболее эффективным, благодаря лидирующим позициям в сфере *Data Science*, математического моделирования и автоматизации лабораторных процессов. Язык обладает встроенными механизмами для работы с последовательными портами и высокоуровневыми структурами данных.

Фреймворк *PyQt5* обоснован следующими преимуществами:

- поддержка паттерна проектирования *MVC*, позволяющего разделить логику обработки физических данных и логику их визуального представления;

- наличие высокоточных таймеров *QTimer*, необходимых для отрисовки графических примитивов с частотой кадров более 30 FPS без блокировки основного потока приложения, принимающего телеметрию;

- нативная интеграция с математической библиотекой *Matplotlib* через класс *FigureCanvasQTAgg*, что позволяет встраивать профессиональные аналитические графики непосредственно в окна приложения.

Для организации подсистемы хранения экспериментальных данных рассматривались реляционные базы данных и плоские текстовые файлы. Развертывание реляционной СУБД для локального сбора телеметрических данных является избыточным архитектурным решением, усложняющим установку и эксплуатацию комплекса.

Наиболее эффективным форматом для хранения массивов измерительных данных является CSV. Использование файлов CSV дает два существенных преимущества:

- исключительно высокая скорость добавления новых записей, что критично при потоковой записи телеметрии;

- идеальная совместимость с библиотекой *pandas*.

Применение библиотеки *pandas* в модуле анализа кардинально упрощает процесс агрегации. Структура *DataFrame* позволяет одной строкой кода выполнять группировку данных по типам диэлектрических преград и вычислять среднее арифметическое значение СВЧ-энергии. Вынесение ресурсоемких операций парсинга и агрегации в изолированный логический модуль гарантирует отзывчивость графического интерфейса даже при работе с масштабными базами данных.

Проектируемый пользовательский интерфейс разделяется на логические зоны. Левая панель отводится под динамическое меню навигации, автоматически масштабируемое на основе уникальных записей в CSV-файле. Правая панель предназначена для визуализации: верхняя часть отдается под гистограмму поглощения, нижняя часть разделяется между таблицей сырых данных и модулем живой симуляции СВЧ-луча. Дополнительно в интерфейс закладывается панель автоматической аналитики, формирующая текстовое заключение о свойствах преграды на основе вычисленных усредненных значений энергии.

2.6 Основные результаты проектирования

На этапе проектирования распределенного аппаратно-программного измерительного комплекса была детально проработана архитектура аппаратного узла на базе микроконтроллера *ESP32 DevKitC V4*. Особое внимание при проектировании было уделено аппаратной изоляции СВЧ-антенны и обеспечению электробезопасности периферийных шин. Предложенные схемотехнические решения, включающие гальваническое разделение силовых линий, установку сглаживающего конденсатора и внедрение защитного резистора в шину данных, полностью исключили риск возникновения

паразитного обратного питания и повреждения логических портов микроконтроллера при отключении внешнего источника энергии.

Платформа *Arduino IDE* и языка программирования *C++* для реализации микропрограммного обеспечения, позволила интегрировать оптимизированные библиотеки для параллельного управления адресной светодиодной индикацией и высокоскоростным аппаратным интерфейсом *UART2* на скорости 256000 бод. Разработка уникального алгоритма неблокирующего опроса на базе прерываний таймера и метода пространственной фильтрации на уровне микроконтроллера успешно решила проблему переполнения буферов обмена. Данный алгоритм аппаратно отсекает ложные эхо-сигналы в ближней зоне сканирования, компенсируя ограничения встроенного цифрового процессора радара *HLK-LD2410C*.

Python в связке с индустриальным фреймворком *PyQt5* это инструмент для создания отказоустойчивого графического интерфейса пользователя с аппаратным ускорением отрисовки динамических анимаций и нативной интеграцией математических плоттеров. Файловая база данных в формате *CSV* в синергии с аналитической библиотекой *Pandas* обеспечивает максимальную производительность при потоковом чтении, математической агрегации и фильтрации больших массивов телеметрии. Спроектированная программная архитектура, основанная на паттерне *Model-View-Controller*, гарантирует строгую модульность приложения, обеспечивая возможность дальнейшего расширения функционала предиктивной аналитики без вмешательства в низкоуровневую логику работы с измерительным оборудованием.

Комплексное применение всех описанных проектных решений формирует надежную и сбалансированную базу для построения измерительного лабораторного стенда. Выбранный гибридный подход к обработке радиолокационных данных, при котором микроконтроллер берет на себя жесткий аппаратный контроль физических процессов, а десктопное приложение обеспечивает глубокую математическую аналитику, исключает появление узких мест в производительности системы. Сформированная аппаратно-программная архитектура полностью отвечает поставленным техническим требованиям и готова к этапу непосредственной практической реализации, написанию исходного кода и последующему проведению серии натурных экспериментов по исследованию затухания сверхвысокочастотных волн.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ХАРАКТЕРИСТИКИ ПРОГРАММНО-АППАРАТНОГО КОМПЛЕКСА

3.1 Структура программного обеспечения и диаграммы взаимодействия

Программная часть измерительного комплекса реализована на двух уровнях: микропрограммном (C++, платформа *ESP32*) и прикладном (*Python*, десктопное приложение). Основной задачей микропрограммного обеспечения является первичная цифровая фильтрация сырых радиолокационных данных и управление аппаратным таймингом, в то время как прикладное приложение отвечает за агрегацию, сохранение и графическую визуализацию результатов эксперимента.

Архитектура клиентского десктопного приложения спроектирована на основе фреймворка *PyQt5* с применением концепции *MVC*. Исходный код разбит на три логических модуля:

- *data_engine.py* для работы с базой данных *CSV* и математическая агрегация с помощью библиотеки *pandas*;
- *radar_canvas.py* кастомный графический движок на базе *QPainter* для симуляции физического распространения СВЧ-луча;
- *main_app.py* контроллер графического интерфейса.

Для формализации функциональных требований и описания логики взаимодействия между аппаратными и программными подсистемами в работе применены стандарты визуального моделирования *UML*.

Функциональные возможности комплекса с точки зрения конечного пользователя формализованы на диаграмме вариантов использования. Основным актором выступает оператор, взаимодействующий с десктопным приложением.

Ключевые сценарии использования включают:

- настройку параметров физической преграды, дистанционную инициализацию аппаратного триггера (посредством встроенных датчиков движения радара);
- анализ визуализированных данных, удаление аномальных замеров из базы данных и экспорт аналитических графиков.

Диаграмма вариантов использования приведена в Приложении А.

Для отражения динамического взаимодействия между программными и аппаратными подсистемами во времени разработана диаграмма последовательности для базового сценария проведения СВЧ-замера. Данная диаграмма описывает строгую хронологию передачи управляющих сигналов и телеметрических данных между графическим интерфейсом *Python*, микроконтроллером *ESP32* и радарным датчиком *HLK-LD2410C*.

Взаимодействие компонентов осуществляется строго асинхронно. Десктопное приложение находится в режиме пассивного прослушивания последовательного порта, в то время как микроконтроллер выступает

инициатором транзакций. Подобная архитектура предотвращает блокировку графического интерфейса пользователя во время выдерживания аппаратных пауз, необходимых для физической эвакуации оператора из зоны распространения микроволн. Диаграмма последовательности приведена в Приложении А.

3.2 Логическая и физическая структура базы данных

Важным этапом реализации лабораторного комплекса является проектирование подсистемы хранения результатов измерений. Ввиду необходимости записи высокочастотных потоковых телеметрических данных и последующей интеграции с математическими библиотеками языка *Python*, в качестве хранилища выбрана плоская файловая база данных в формате CSV.

В отличие от классических клиент-серверных реляционных СУБД, требующих выполнения ресурсоемких *SQL*-запросов и поддержания сетевого соединения, формат CSV обеспечивает скорость операций ввода-вывода на уровне файловой системы операционной системы. При этом логическая структура CSV-файла удовлетворяет требованиям первой нормальной формы теории реляционных баз данных: каждая запись имеет уникальный идентификатор, значения атрибутов атомарны, а скрытые зависимости отсутствуют.

В предметной области измерительного комплекса выделена одна основная сущность – СВЧ_Замер. Логическая модель данных представлена на рисунке 3.1.

radar_physics_data		
Integer	Эксперимент	PK
String	Материал	
Float	е_Проницаемость	
Integer	Толщина_мм	
Float	Дистанция_см	
Float	Энергия	

Рисунок 3.1 – Логическая модель данных

Физическая модель данных состоит из следующих атрибутов, записываемых *Python*-приложением после каждого успешного цикла сканирования:

- эксперимент – уникальный целочисленный идентификатор замера, генерируемый приложением автоматически путем инкрементирования максимального существующего номера в базе;
- материал – строковое наименование исследуемого диэлектрика;
- ϵ Проницаемость – десятичное число с плавающей точкой, отражающее справочное значение относительной диэлектрической проницаемости (ϵ) исследуемого материала;
- толщина_мм – целое число, физический габарит преграды, характеризующий длину пути прохождения СВЧ-луча сквозь среду;
- дистанция_см – десятичное число, математически усредненное значение дальности до эталонной мишени, рассчитанное на основе четырех сырых пакетов данных от радара;
- энергия – десятичное число, усредненный безразмерный индекс спектральной плотности отраженного сигнала, являющийся ключевым параметром для оценки диэлектрического затухания электромагнитной волны.

Для загрузки, фильтрации и агрегации данных из физической модели в оперативную память десктопного приложения используется структура *DataFrame*. Обновление пользовательского интерфейса происходит посредством вызова встроенных методов группировки, которые сортируют записи по атрибуту материала и вычисляют среднее значение атрибута энергии. Данный подход обеспечивает целостность исторических данных и высокую скорость аналитической обработки без необходимости развертывания локального SQL-сервера.

3.3 Графический интерфейс пользователя и визуализация процессов

Для приема, агрегации и визуализации телеметрических данных разработано десктопное приложение на языке *Python*. В качестве основы графического интерфейса пользователя (*GUI*) применен фреймворк *PyQt5*, обеспечивающий высокую производительность отрисовки элементов и отсутствие блокировок основного потока программы при асинхронном ожидании данных от микроконтроллера.

Структурно главное окно программного комплекса разделено на три изолированные функциональные зоны. Панель навигации и управления, размещенная в левой части окна, отличается механизмом динамической генерации навигационных кнопок: модуль аналитики сканирует базу данных *CSV* и автоматически создает фильтры для каждого уникального исследованного материала. Нажатие на кнопку инициирует мгновенную фильтрацию телеметрии во всем приложении и визуальное выделение результатов на графиках.

Аналитическая зона, занимающая верхнюю правую часть интерфейса, включает интерактивную гистограмму, интегрированную посредством библиотеки *Matplotlib*. Данный график в реальном времени отображает спектр поглощения диэлектриков, демонстрируя усредненную мощность отраженного сигнала для каждого протестированного материала, при этом для нужд документирования предусмотрена функция аппаратного экспорта аналитического графика в растровый формат *PNG* высокого разрешения. Непосредственно под графиком

располагается панель интеллектуального вывода, формирующая физическое экспертное заключение на основе уровня полученной СВЧ-энергии путем классификации преграды как радиопрозрачной среды, материала со средним поглощением или абсолютного радиоэкрана.

Мониторинговая зона, занимающая нижнюю половину экрана, функционально разделена на журнал сырых данных (*QTableWidget*) для хронологического контроля дисперсии значений дистанции и энергии, а также модуль физической симуляции. Разработанный графический движок симуляции, использующий низкоуровневый класс *QPainter*, генерирует анимацию распространения электромагнитного поля, которая в зависимости от уровня принятой энергии наглядно демонстрирует либо беспрепятственное прохождение концентрических СВЧ-волн сквозь радиопрозрачную преграду, либо жесткую блокировку волнового фронта.

Внешний вид разработанного десктопного приложения в режиме анализа представлен на рисунке 3.2.

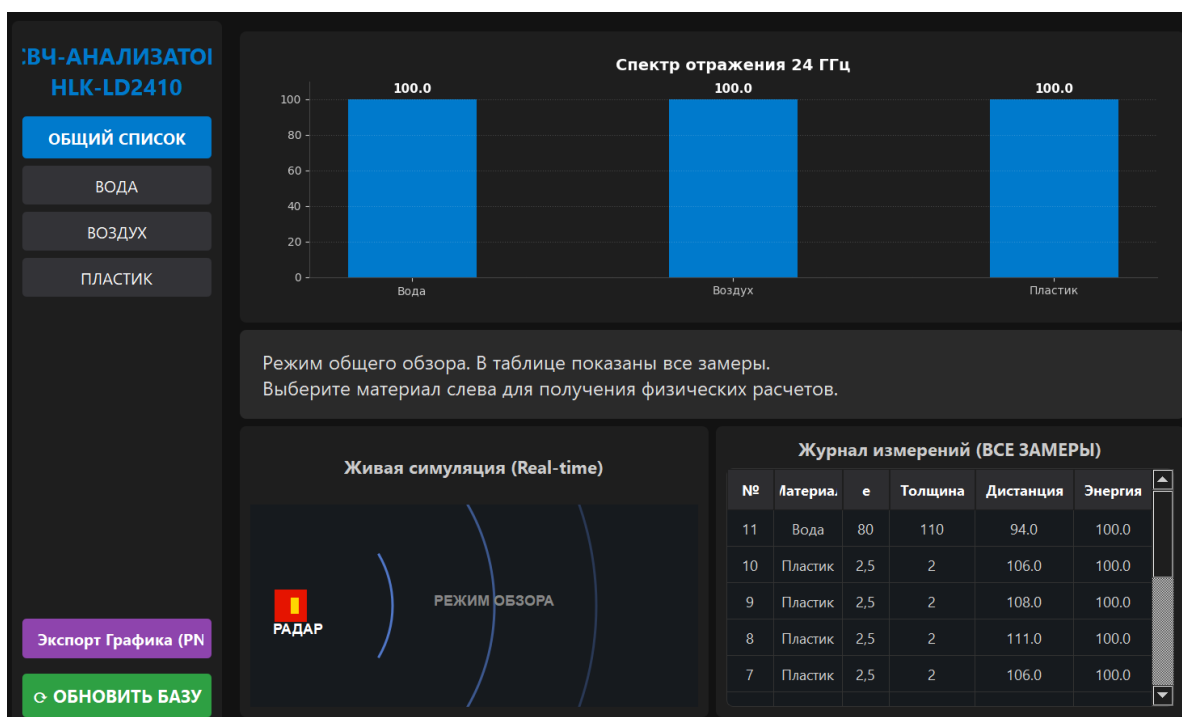


Рисунок 3.2 – Графический интерфейс лабораторного программного комплекса СВЧ-анализа

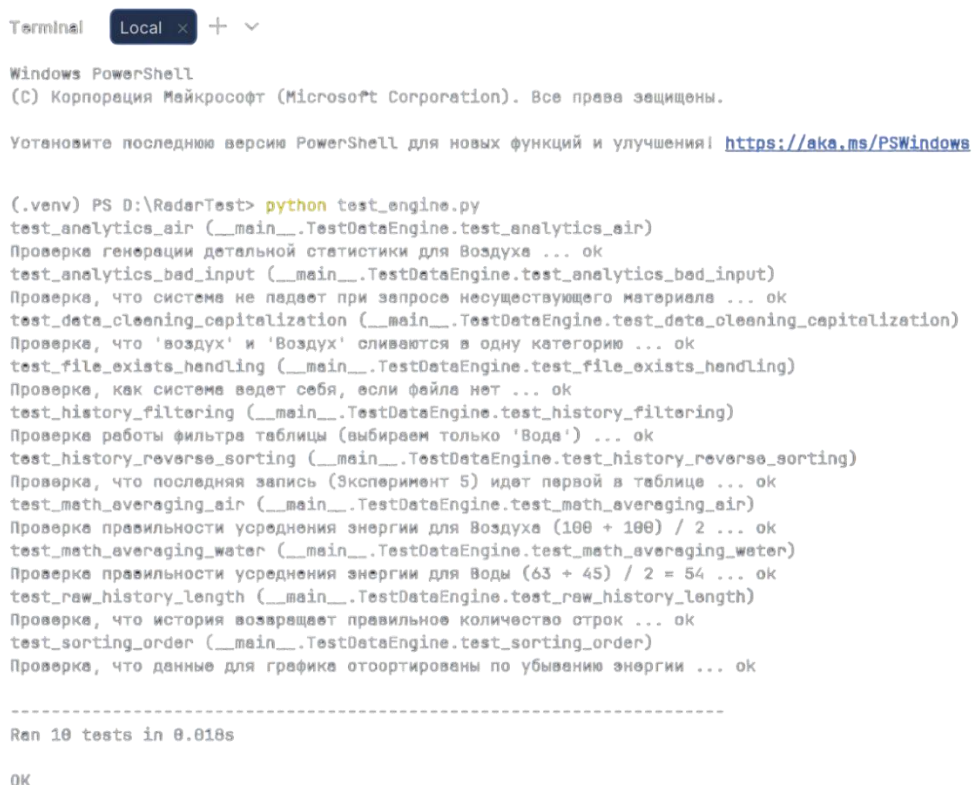
Для подтверждения надежности, отказоустойчивости и вычислительной эффективности разработанного аппаратно-программного комплекса было проведено комплексное тестирование. Основной акцент при проверке программного обеспечения верхнего уровня был сделан на корректность математической обработки телеметрии и скорость взаимодействия с файловой базой данных.

Оценка логики работы ядра приложения (*data_engine.py*) выполнена с помощью автоматизированных модульных тестов (*Unit-тестов*) на базе встроенной

библиотеки *unittest*. Тестирование инкапсулировано в изолированном классе *TestDataEngine*. В целях предотвращения искажения экспериментальных данных, алгоритм тестирования предусматривает автоматическое создание и последующее удаление временного файла-заглушки (*Mock*-объекта), имитирующего структуру реальной базы данных CSV.

В ходе автоматизированного тестирования верификации подвергся ряд критических узлов программного обеспечения. Алгоритм проверки охватил корректность обработки системных исключений при попытках инициализации без файлов хранилища, а также надежность механизмов очистки и нормализации строковых данных. Особое внимание было уделено математической точности вычисления среднего арифметического значения СВЧ-энергии для различных типов диэлектриков. Дополнительно тестировалась корректность *LIFO*-сортировки и фильтрации массивов данных для виджета журнала измерений, стабильность работы подсистемы автоматической аналитики и встроенные алгоритмы защиты приложения от невалидных пользовательских запросов.

Результат запуска пакета автоматических модульных тестов через встроенный терминал среды разработки приведен на рисунке 3.2.



```
Terminal Local + -
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшений! https://aka.ms/PSWindows

(.venv) PS D:\RadarTest> python test_engine.py
test_analytics_air (__main__.TestDataEngine.test_analytics_air)
Проверка генерации детальной статистики для Воздуха ... ok
test_analytics_bad_input (__main__.TestDataEngine.test_analytics_bad_input)
Проверка, что система не падает при запросе несуществующего материала ... ok
test_data_cleaning_capitalization (__main__.TestDataEngine.test_data_cleaning_capitalization)
Проверка, что 'воздух' и 'Воздух' сливаются в одну категорию ... ok
test_file_exists_handling (__main__.TestDataEngine.test_file_exists_handling)
Проверка, как система ведет себя, если файла нет ... ok
test_history_filtering (__main__.TestDataEngine.test_history_filtering)
Проверка работы фильтра таблицы (выбираем только 'Вода') ... ok
test_history_reverse_sorting (__main__.TestDataEngine.test_history_reverse_sorting)
Проверка, что последняя запись (Эксперимент 5) идет первой в таблице ... ok
test_math_averaging_air (__main__.TestDataEngine.test_math_averaging_air)
Проверка правильности усреднения энергии для Воздуха (100 + 100) / 2 ... ok
test_math_averaging_water (__main__.TestDataEngine.test_math_averaging_water)
Проверка правильности усреднения энергии для Воды (63 + 45) / 2 = 54 ... ok
test_raw_history_length (__main__.TestDataEngine.test_raw_history_length)
Проверка, что история возвращает правильное количество строк ... ok
test_sorting_order (__main__.TestDataEngine.test_sorting_order)
Проверка, что данные для графика отсортированы по убыванию энергии ... ok

-----
Ran 10 tests in 0.018s

OK
```

Рисунок 3.3 – Результаты выполнения пакета автоматизированных модульных тестов

Как видно из результатов выполнения скрипта, все 10 разработанных тест-кейсов завершились успешно. Важным показателем оценки производительности является время отклика системы. Общее время выполнения пула агрегирующих и математических операций над базой данных составило 0,018 с. Столь высокий

показатель быстродействия доказывает целесообразность выбранного архитектурного решения (интеграция файлов *CSV* и библиотеки *Pandas*).

Анализ результатов тестирования подтверждает высокое качество исходного кода. Desktopное приложение устойчиво к программным исключениям, обеспечивает мгновенную обработку массивов телеметрии и полностью соответствует заявленным требованиям для проведения высокоточных радиофизических измерений.

3.4 Конфигурация аппаратных зон чувствительности СВЧ-радара

Первичная настройка и калибровка цифрового сигнального процессора радара *HLK-LD2410C* является критическим этапом подготовки лабораторного комплекса. Ввиду широкой диаграммы направленности антенны (угол раскрытия $\approx 60^\circ$), радар способен фиксировать паразитные переотражения от элементов полигона и биологического фона оператора в ближней зоне. Для устранения аппаратной интерференции применяется функция пространственного зонирования чувствительности. Доступ к внутренним регистрам памяти модуля осуществляется посредством протокола *Bluetooth Low Energy* через специализированное сервисное мобильное приложение *HLKRadarTool*.

В архитектуре датчика рабочее пространство разделено на дистанционные врата (*Gates*) с фиксированным шагом 0,75 метра. Для обеспечения чистоты эксперимента и подавления ближней зоны параметр статической чувствительности для зоны 0 (0-75 см) программно загрублен до максимального значения. Подобная конфигурация заставляет *DSP*-процессор полностью игнорировать любые неподвижные электромагнитные отклики в радиусе до 75 сантиметров. Это фактически ослепляет радар вблизи, запрещая ему фокусироваться на тестируемой диэлектрической преграде и позволяя СВЧ-лучу беспрепятственно сканировать пространство за ней.

Для обеспечения максимальной восприимчивости приемного тракта к отраженному сигналу от эталонной мишени, установленной на дистанции 100 см, статическая чувствительность для зоны 1 установлена на высокий уровень (значение 15). Максимальная дальность зондирования аппаратно ограничена значением 1,5 метра (2 зоны) с целью исключения фиксации ложных эхо-сигналов от дальних стен и мебели помещения.

Критически важной настройкой является блокировка встроенного алгоритма адаптивного вычитания фона. Для предотвращения программного удаления абсолютно неподвижной мишени в категорию фоновых шумов в регистр *Unmanned duration* (задержка выключения) записано предельно допустимое значение, равное 60 секундам. Это гарантирует удержание эталонного отражателя в буфере *DSP*-процессора на протяжении всего цикла сканирования и маневров оператора.

Интерфейс сервисного приложения *HLKRadarTool* с выставленными конфигурационными параметрами представлен на рисунке 3.4.

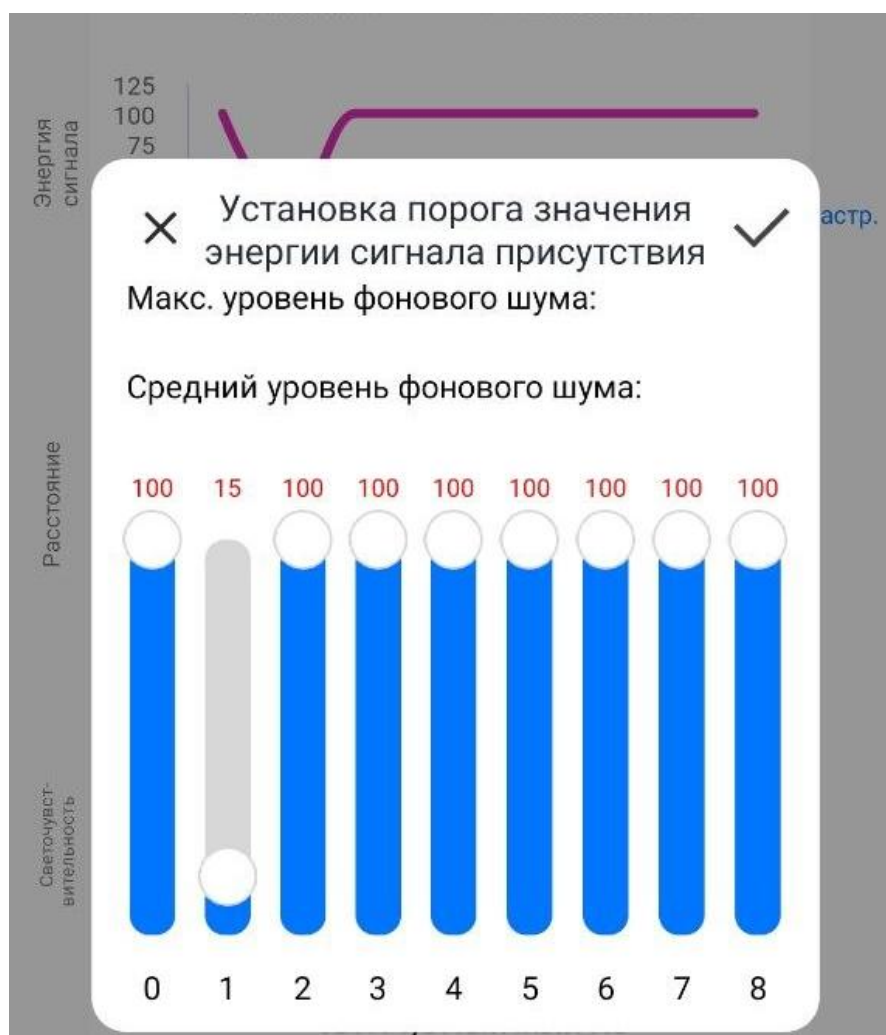


Рисунок 3.4 – Конфигурирование дистанционных врат в приложении *HLKRadarTool*

Для обеспечения максимальной восприимчивости приемного тракта к отраженному сигналу от эталонной мишени, установленной на дистанции 100 см, статическая чувствительность для зоны 1 установлена на высокий уровень (значение 15). Максимальная дальность зондирования аппаратно ограничена значением 1,5 метра (2 зоны) с целью исключения фиксации ложных эхо-сигналов от дальних стен и мебели помещения.

Критически важной настройкой является блокировка встроенного алгоритма адаптивного вычитания фона. Для предотвращения программного удаления абсолютно неподвижной мишени в категорию фонового шума в регистр *Unmanned duration* записано предельно допустимое значение, равное 60 секундам. Это гарантирует удержание эталонного отражателя в буфере *DSP*-процессора на протяжении всего цикла сканирования и маневров оператора.

Как видно на графике настройки параметров присутствия, пороговое значение чувствительности для нулевых врат (отметка 0 на оси абсцисс) установлено на максимальный уровень (100), что программно формирует «слепую

зону». Для первых врат (отметка 1) порог снижен до 15, образуя окно максимальной восприимчивости для фиксации эталонной мишени. Остальные дистанционные врата программно заблокированы. Также в настройках зафиксирована требуемая скорость передачи данных по интерфейсу *UART*.

Интерфейс сервисного приложения *HLKRadarTool* с выставленными конфигурационными параметрами представлен на рисунке 3.5.



Рисунок 3.5 – Конфигурирование дистанционных врат в приложении *HLKRadarTool*

3.5 Результаты экспериментов и оценка диэлектрического затухания СВЧ-сигнала

Завершающим этапом практической реализации комплекса стало проведение серии натурных экспериментов, направленных на оценку степени затухания СВЧ-излучения при прохождении через среды с различной диэлектрической проницаемостью. Все эксперименты проводились с использованием металлической эталонной мишени, установленной на дистанции 108 см от радара. Выбор металла обусловлен его свойствами абсолютного СВЧ-отражателя.

В первой серии экспериментов проводилась калибровка базовой линии. Роль среды распространения выполнял атмосферный воздух ($\epsilon \approx 1$). Ввиду отсутствия поглощающих факторов, электромагнитная волна беспрепятственно достигала мишени. Desktopное приложение стабильно фиксировало индекс отраженной энергии на максимальной отметке (100 условных единиц).

Во второй серии экспериментов на отметке 50 см (строго по центру оси излучения) устанавливалась преграда из полиэтилентерефталата (пустая ПЭТ-бутылка). Пластик относится к классу радиопрозрачных диэлектриков ($\epsilon \approx 2,5$). Результаты измерений показали, что СВЧ-сигнал проходит сквозь данную преграду практически без ослабления амплитуды. Уровень отраженной от мишени энергии сохранился на максимальной отметке (100 единиц). Незначительные потери энергии на френелевское отражение от стенок бутылки оказались ниже порога чувствительности АЦП радара, что подтверждает возможность использования пластиковых корпусов для скрытого монтажа подобных датчиков телеметрии.

В ходе отдельного калибровочного теста эталонная металлическая мишень была помещена внутрь широкого пластикового резервуара (таза). Несмотря на наличие дополнительных стенок резервуара, радарный комплекс безошибочно регистрировал микро-движения мишени и стабильно возвращал уровень энергии 100 единиц. Данный факт подтверждает, что радиопрозрачные материалы не вносят критических искажений в фазу отраженного сигнала, позволяя алгоритму *FMCW* корректно вычислять дистанцию.

Ключевой серией экспериментов стала оценка радиопоглощающих свойств воды. В качестве преграды на отметке 50 см использовалась та же ПЭТ-бутылка, полностью заполненная водой ($\epsilon \approx 80$). Вода, являясь сильно полярным диэлектриком, выступает в роли эффективного СВЧ-щита для частоты 24 ГГц. Энергия падающей электромагнитной волны затрачивается на дипольную релаксацию молекул воды, необратимо преобразуясь в тепловую энергию.

Результаты измерений продемонстрировали резкое падение мощности отраженного от мишени сигнала. Уровень регистрируемой энергии снизился с эталонных 100 единиц до диапазона 40-70 условных единиц. Подобное существенное затухание амплитуды является прямым физическим доказательством радиопоглощающих свойств воды.

Отдельного аналитического рассмотрения требует наблюдаемая дисперсия значений энергии в эксперименте с водой (скачки в диапазоне 40-70 единиц). Наличие остаточной энергии (отличной от нуля) при использовании бутылки обусловлено явлением радиоволновой дифракции. Угол раскрытия антенны радара составляет $\approx 60^\circ$, в результате чего ширина фронта СВЧ-луча на отметке 50 см значительно превышает диаметр бутылки.

Центральная часть волнового фронта полностью блокировалась и поглощалась объемом воды, однако боковые "лепестки" излучения огибали преграду, отражались от стен помещения, достигали металлической мишени и возвращались в приемную антенну радара. Данный физический эффект носит название многолучевого распространения. Волны, пришедшие в приемник различными (окольными) путями, имеют разные фазы. Их интерференция (сложение) в точке

приема приводит к флуктуациям амплитуды результирующего сигнала, что программно фиксировалось комплексом как скачкообразное изменение индекса энергии.

Проведенный комплекс экспериментов не только подтвердил заявленные теоретические физические модели, но и доказал высокую эффективность разработанного программно-аппаратного комплекса для регистрации, фильтрации и визуализации сложных эффектов распространения СВЧ-излучения.

Успешная регистрация таких тонких радиофизических явлений, как дифракция и многолучевая интерференция, подтверждает абсолютную корректность заложенных в приложение алгоритмов цифровой обработки и математической агрегации телеметрии. Созданный измерительный стенд продемонстрировал высокую чувствительность и разрешающую способность, достаточную для проведения базового неразрушающего контроля материалов. Полученные эмпирические результаты не только имеют высокую ценность для образовательного процесса в рамках наглядного изучения радиофизики, но и открывают широкие перспективы для адаптации комплекса под промышленные задачи или экосистемы интернета вещей. В частности, подтвержденная способность радара игнорировать радиопрозрачные преграды делает разработанную аппаратную архитектуру идеальным решением для систем скрытого мониторинга и автоматизации помещений, где требуется точное бесконтактное детектирование присутствия людей сквозь оптически непрозрачные элементы интерьера.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта был успешно спроектирован, реализован и протестирован распределенный программно-аппаратный комплекс для телеметрического исследования характеристик затухания СВЧ-излучения на частоте 24 ГГц. Поставленные цели и задачи исследования были выполнены в полном объеме.

Проведен аналитический обзор физических принципов радиолокации в К-диапазоне. Теоретически обоснованы радиопоглощающие свойства полярных диэлектриков и радиопрозрачность материалов с низкой диэлектрической проницаемостью. Выбор технологии непрерывного частотно-модулированного излучения на базе радарного датчика *HLK-LD2410C* доказал свою эффективность для задач высокоточного бесконтактного позиционирования.

Спроектирована и смонтирована аппаратная часть измерительного полигона на базе высокопроизводительного микроконтроллера *ESP32 DevKitC V4*. Успешно решена критическая проблема паразитного обратного питания информационных шин, возникающая при асинхронном обесточивании периферийных модулей. Внедрение токоограничивающих резисторов, сглаживающих емкостей и разработка строгого регламента коммутации напряжений обеспечили абсолютную отказоустойчивость и электробезопасность аппаратного узла.

Разработано оптимизированное микропрограммное обеспечение на языке C++. Реализован событийный алгоритм управления с гибридным триггером. Применение механизма неблокирующих задержек с параллельной очисткой *UART*-буфера полностью устранило проблему потери пакетов телеметрии на высоких скоростях обмена. Интегрированный программный фильтр пространственной селекции позволил аппаратно отсекал ложные эхо-сигналы в ближней зоне (до 75 см), компенсируя ограничения встроенного цифрового процессора радара.

Для компенсации алгоритма адаптивного подавления статического фона, встроенного в *DSP*-процессор радара, разработана и апробирована методика «удаленного микро-пинга». Дистанционное приведение эталонной металлической мишени в состояние микро-вибрации обеспечило идеальную чистоту радиоэфира и стабильную регистрацию СВЧ-откликов.

Спроектировано и внедрено кроссплатформенное десктопное приложение на языке *Python* с использованием фреймворка *PyQt5*. Архитектура приложения, построенная по паттерну *MVC*, обеспечила надежное разделение логики работы с файловой базой данных и модулями визуализации. Использование библиотек *Pandas* и *Matplotlib* позволило реализовать функционал агрегации телеметрии, расчета затухания сигнала и построения аналитических гистограмм в реальном времени.

Разработан уникальный графический модуль живой симуляции распространения электромагнитных волн, наглядно демонстрирующий эффекты прохождения СВЧ-луча сквозь радиопрозрачные среды и его блокировку при взаимодействии с поглощающими преградами. Внедрен аналитический модуль

Список использованных источников

1. Баскаков, С. И. Радиотехнические цепи и сигналы : учебник для вузов / С. И. Баскаков. – 5-е изд., перераб. и доп. – М. : Высшая школа, 2005. – 462 с.
2. Никольский, В. В. Электродинамика и распространение радиоволн : учебное пособие / В. В. Никольский, Т. И. Никольская. – 6-е изд. – М. : Наука, 1989. – 543 с.
3. Доусон, М. Изучаем C++ через программирование игр / М. Доусон ; пер. с англ. – СПб. : Питер, 2016. – 416 с.
4. Лутц, М. Изучаем Python / М. Лутц ; пер. с англ. – 4-е изд. – СПб. : Символ-Плюс, 2011. – 1280 с.
5. Маккинни, У. Python и анализ данных / У. Маккинни ; пер. с англ. – М. : ДМК Пресс, 2015. – 482 с.
6. Прохоренок, Н. А. Python 3 и PyQt 5. Разработка приложений / Н. А. Прохоренок, В. А. Дронов. – СПб. : БХВ-Петербург, 2021. – 832 с.
7. Вандер Плас, Дж. Python для сложных задач. Наука о данных и машинное обучение / Дж. Вандер Плас ; пер. с англ. – СПб. : Питер, 2018. – 57 с.
8. Espressif IoT Development Framework (ESP-IDF) : официальная документация [Электронный ресурс]. – Режим доступа : <https://docs.espressif.com/project>. – Дата доступа : 10.04.2026.
9. Arduino Reference : официальный справочник по языку [Электронный ресурс]. – Режим доступа : <https://www.arduino.cc/reference/en/>. – Дата доступа : 10.04.2026.
10. Pandas : официальная документация библиотеки [Электронный ресурс]. – Режим доступа : <https://pandas.pydata.org/docs/>. – Дата доступа : 10.04.2026.
11. Matplotlib : официальная справочная система [Электронный ресурс]. – Режим доступа : <https://matplotlib.org/stable/contents.html>. – Дата доступа : 10.04.2026.
12. PyQt5 Reference Guide : официальное руководство [Электронный ресурс]. – Режим доступа : <https://www.riverbankcomputing.com/static/Docs/PyQt5/>. – Дата доступа : 10.04.2026.
13. HLK-LD2410C Datasheet : техническая спецификация радарного модуля / Hi-Link Electronic Co., Ltd [Электронный ресурс]. – Режим доступа : <https://hlktech.net/>. – Дата доступа : 10.04.2026.
14. Фримен, Э. Head First. Паттерны проектирования / Э. Фримен, Э. Робсон ; пер. с англ. – 2-е изд., обновл. – СПб. : Питер, 2022. – 656 с..

ПРИЛОЖЕНИЕ А

(обязательное)

Диаграммы эксплуатируемого ПО

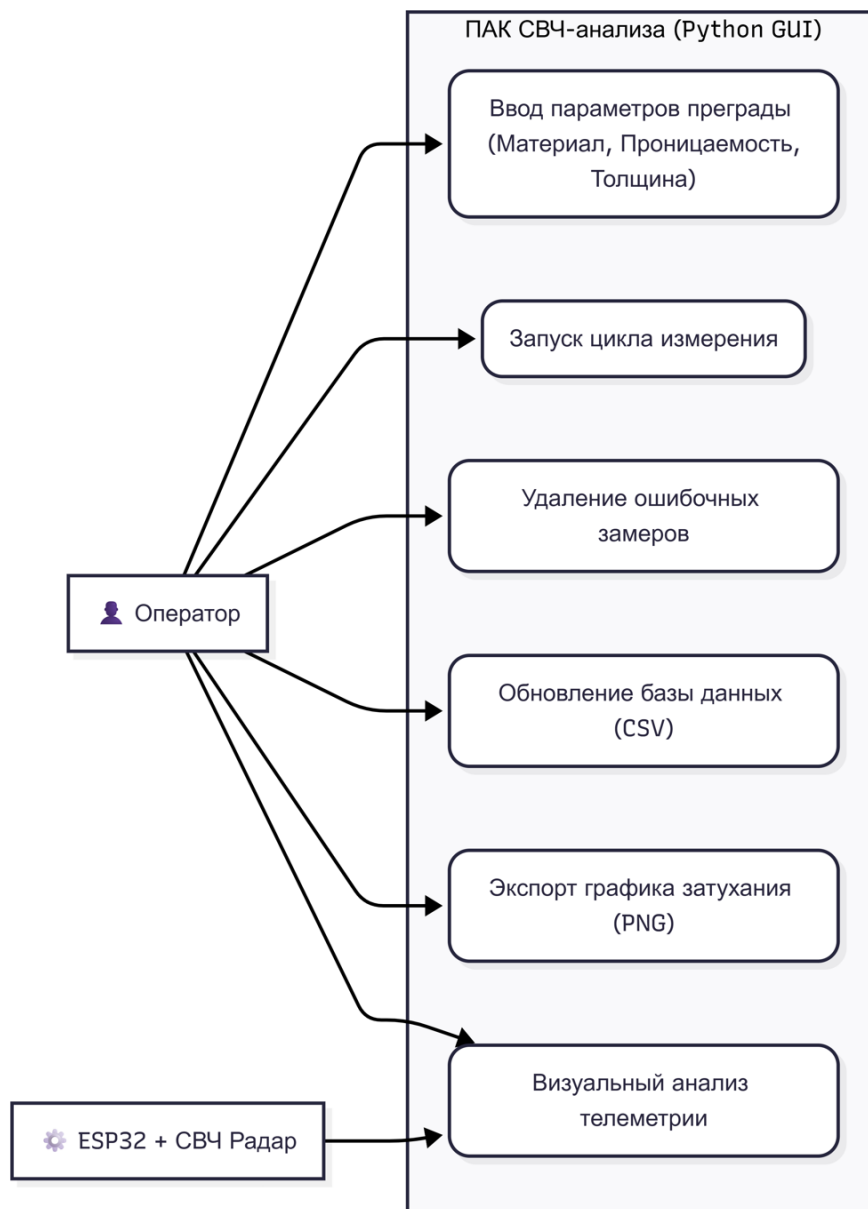


Рисунок А.1 – Диаграмма вариантов использования

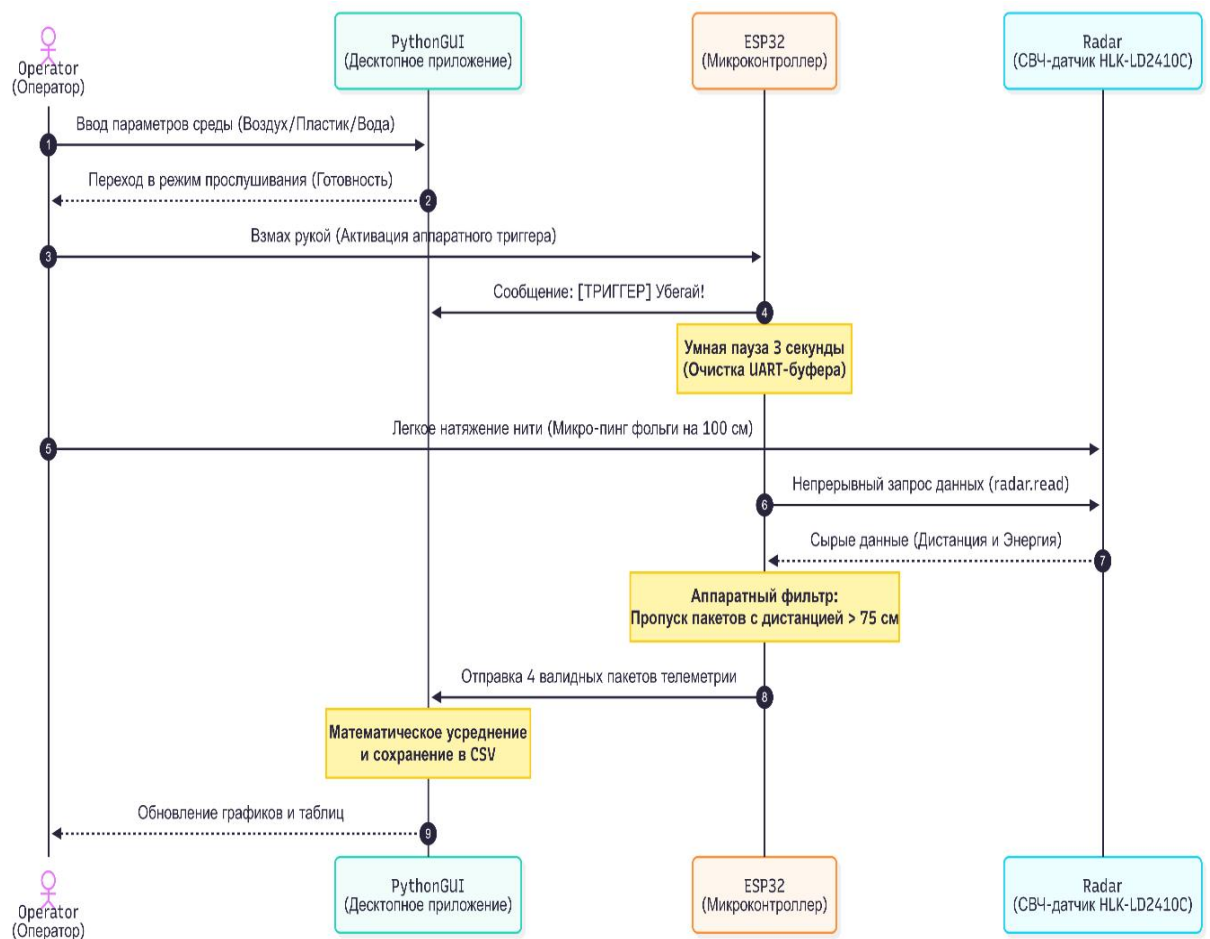


Рисунок А.2 – Диаграмма последовательности

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программы

sketch_mar23a:

```
#include <Adafruit_NeoPixel.h>
#include <ld2410.h>

#define LED_PIN 15
#define NUMPIXELS 30

Adafruit_NeoPixel pixels(NUMPIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);
ld2410 radar;

void setup() {
  Serial.begin(115200);
  pixels.begin();
  pixels.setBrightness(20);
  pixels.clear(); pixels.show();

  Serial2.begin(256000, SERIAL_8N1, 16, 17);
  radar.begin(Serial2);

  Serial.println("=== РЕЖИМ 'ВСЕЯДНЫЙ ШАЙПЕР' (ДВИЖЕНИЕ + СТАТИКА) ГОТОВ
===");
}

void loop() {
  radar.read();

  if (radar.isConnected()) {

    // 1. АКТИВАЦИЯ (Твой взмах рукой в Зоне 0)
    if (radar.movingTargetEnergy() > 40) {

      pixels.clear();
      for(int i=0; i<5; i++) pixels.setPixelColor(i, pixels.Color(0, 0, 255));
      pixels.show();

      Serial.println("\n[ТРИГГЕР] Махнул. Убегай!");

      // 2. ДАЕМ 3 СЕКУНДЫ НА ПОБЕГ
      uint32_t t = millis();
      while(millis() - t < 3000) { radar.read(); }

      Serial.println("[ЗАМЕР] Ищу любую цель дальше 75 см...");

      pixels.clear();
      for(int i=0; i<5; i++) pixels.setPixelColor(i, pixels.Color(255, 255, 0));
      pixels.show();
    }
  }
}
```

```

int count = 0;

// 3. СКАНИРУЕМ КОМНАТУ (30 попыток)
for (int i = 0; i < 30; i++) {
    radar.read();

    int target_dist = 0;
    int target_energy = 0;

    // ПРОВЕРЯЕМ СНАЧАЛА ДВИЖЕНИЕ (Вибрацию телефона)
    if (radar.movingTargetDetected() && radar.movingTargetDistance() >= 75) {
        target_dist = radar.movingTargetDistance();
        target_energy = radar.movingTargetEnergy();
    }
    // ЕСЛИ ДВИЖЕНИЯ НЕТ, ПРОВЕРЯЕМ СТАТИКУ (Если фольга замерла)
    else if (radar.stationaryTargetDetected() && radar.stationaryTargetDistance() >= 75) {
        target_dist = radar.stationaryTargetDistance();
        target_energy = radar.stationaryTargetEnergy();
    }

    // 4. ЕСЛИ НАШЛИ ХОТЬ ЧТО-ТО ДАЛЬШЕ 75 см -> ПЕЧАТАЕМ
    if (target_dist >= 75) {
        Serial.print("ЗАМЕР | Дистанция: ");
        Serial.print(target_dist);
        Serial.print(" см | Энергия: ");
        Serial.println(target_energy);

        count++;
        if (count >= 4) break; // Собрали 4 строчки - выходим
    }
    delay(100);
}

if(count == 0) {
    Serial.println("Цель не найдена...");
}

pixels.clear(); pixels.show();

// Глухая пауза 3 сек перед новым тестом
t = millis();
while(millis() - t < 3000) { radar.read(); }
}
}
}

```

radar_logger:

```

import serial
import time
import csv
import os

```

```

COM_PORT = 'COM6' # Проверь свой порт!
BAUD_RATE = 115200
FILE_NAME = 'radar_physics_data.csv'

# --- ФУНКЦИИ ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ ---

def get_next_experiment_id():
    """Читает файл и находит следующий свободный номер эксперимента"""
    if not os.path.exists(FILE_NAME):
        return 1

    max_id = 0
    with open(FILE_NAME, mode='r', encoding='utf-8-sig') as file:
        reader = csv.reader(file, delimiter=';')
        next(reader, None) # Пропускаем заголовки
        for row in reader:
            if row and row[0].isdigit():
                max_id = max(max_id, int(row[0]))
    return max_id + 1

def delete_experiment(exp_id):
    """Удаляет строку с указанным номером эксперимента из CSV"""
    if not os.path.exists(FILE_NAME):
        return False

    rows = []
    deleted = False
    with open(FILE_NAME, mode='r', encoding='utf-8-sig') as file:
        reader = csv.reader(file, delimiter=';')
        header = next(reader, None)
        if header:
            rows.append(header)
        for row in reader:
            if row and row[0] == str(exp_id):
                deleted = True
                continue # Пропускаем эту строку (удаляем)
            rows.append(row)

    if deleted:
        with open(FILE_NAME, mode='w', newline="", encoding='utf-8-sig') as file:
            writer = csv.writer(file, delimiter=';')
            writer.writerows(rows)
    return deleted

# --- ИНИЦИАЛИЗАЦИЯ ---

# Создаем файл с заголовками, если его нет
try:
    if not os.path.exists(FILE_NAME):
        with open(FILE_NAME, mode='w', newline="", encoding='utf-8-sig') as file:
            writer = csv.writer(file, delimiter=';')

```

```

        writer.writerow(['Эксперимент', 'Материал', 'e_Проницаемость',
                        'Толщина(мм)', 'Дистанция(см)', 'Энергия'])
except Exception:
    pass

try:
    ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
    print(f"Подключение к радару на {COM_PORT} УСПЕШНО")
    time.sleep(2)
    ser.reset_input_buffer()
except Exception:
    print("\nОШИБКА СОМ-ПОРТА! Закрой Монитор порта в Arduino IDE!")
    exit()

print("\n" + "="*55)
print("  ЛАБОРАТОРНЫЙ КОМПЛЕКС СВЧ-ИЗЛУЧЕНИЯ ГОТОВ")
print("="*55)

# --- ГЛАВНЫЙ ЦИКЛ ПРОГРАММЫ ---

while True:
    next_id = get_next_experiment_id()
    print(f"\nТекущий номер для записи: [ {next_id} ]")
    print(" [ENTER] - Начать новый замер")
    print(" [ d ] - Удалить ошибочный замер")
    print(" [ q ] - Выйти из программы")

    action = input("Твой выбор: ").strip().lower()

    if action == 'q':
        print("Работа завершена. Данные сохранены.")
        break

    elif action == 'd':
        del_id = input("Введи НОМЕР эксперимента, который нужно удалить: ").strip()
        if del_id.isdigit():
            if delete_experiment(del_id):
                print(f"[*] Эксперимент №{del_id} УСПЕШНО УДАЛЕН из базы!")
            else:
                print(f"[!] ОШИБКА: Эксперимент №{del_id} не найден.")
        else:
            print(f"[!] ОШИБКА: Номер должен быть числом.")
            continue

    elif action == "":
        # Нажали ENTER - начинаем замер!
        exp_name = str(next_id)

        print(f"\n--- НАСТРОЙКА ЭКСПЕРИМЕНТА №{exp_name} ---")
        material = input("Материал (Воздух, Пластик, Вода): ")
        epsilon = input("Проницаемость (1, 2.5, 80): ")
        thickness = input("Толщина (мм): ")

```

```

print("\n[ОЖИДАНИЕ] Махни рукой -> Сядь -> ДЕРГНИ НИТКУ!")

ser.reset_input_buffer()
measurements = []

# СЛУШАЕМ РАДАР
while True:
    if ser.in_waiting > 0:
        try:
            line = ser.readline().decode('utf-8', errors='ignore').strip()

            if "[ТРИГГЕР]" in line:
                print("> Триггер сработал! Дергай нитку...")

            if "ЗАМЕР | Дистанция:" in line:
                parts = line.split('|')
                dist = int(parts[1].replace("Дистанция:", "").replace("см", "").strip())
                energy = int(parts[2].replace("Энергия:", "").strip())

                measurements.append((dist, energy))
                print(f"Выстрел {len(measurements)}/4: Дальность {dist} см, Энергия {energy}")

            # Если собрали 4 замера или радар не нашел цель
            if len(measurements) >= 4 or ("Цель не найдена" in line and len(measurements) > 0):
                time.sleep(0.5)
                break
        except Exception:
            pass

# ЗАПИСЬ РЕЗУЛЬТАТОВ
if len(measurements) > 0:
    avg_dist = round(sum(d for d, e in measurements) / len(measurements), 1)
    avg_energy = round(sum(e for d, e in measurements) / len(measurements), 1)

    print(f"\n[ИТОГ] Мишень: {avg_dist} см | Мощность (P): {avg_energy}/100")

    with open(FILE_NAME, mode='a', newline="", encoding='utf-8-sig') as file:
        writer = csv.writer(file, delimiter=';')
        writer.writerow([exp_name, material, epsilon, thickness, avg_dist, avg_energy])
    print(f"[*] Данные Эксперимента №{exp_name} успешно сохранены в CSV!")
else:
    print("\n[ОШИБКА] Радар не увидел мишень. (Возможно, сигнал заблокирован водой!)")

main_app:
# Файл: main_app.py
import sys
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                              QHBoxLayout, QPushButton, QLabel, QFrame, QTableWidgetItem,

```

```

        QTableWidgetItem, QHeaderView, QAbstractItemView, QMessageBox)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
import matplotlib.pyplot as plt
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas

from data_engine import DataEngine
from radar_canvas import RadarAnimation

BG_APP = "#121212"
BG_PANEL = "#1E1E1E"
TEXT_LIGHT = "#CCCCCC"
ACCENT_BLUE = "#007acc"
ACCENT_RED = "#D32F2F"

class RadarApp(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Лаборатория Радиофизики 2.0 (СВЧ 24 ГГц)")
        self.setGeometry(50, 50, 1400, 900)
        self.setStyleSheet(f"background-color: {BG_APP}; color: {TEXT_LIGHT};")

        self.engine = DataEngine()
        self.current_material = "ВСЕ ЗАМЕРЫ"

        self.init_ui()
        self.refresh_data()

    def init_ui(self):
        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        main_layout = QHBoxLayout(central_widget)

        # --- ЛЕВАЯ ПАНЕЛЬ ---
        left_panel = QFrame()
        left_panel.setFixedWidth(260)
        left_panel.setStyleSheet(f"background-color: {BG_PANEL}; border-radius: 8px;")
        left_layout = QVBoxLayout(left_panel)

        title_lbl = QLabel("СВЧ-АНАЛИЗАТОР\nHLK-LD2410")
        title_lbl.setFont(QFont("Segoe UI", 14, QFont.Bold))
        title_lbl.setAlignment(Qt.AlignCenter)
        title_lbl.setStyleSheet(f"color: {ACCENT_BLUE}; padding: 10px 0px;")
        left_layout.addWidget(title_lbl)

        # Кнопка "ОБЩИЙ СПИСОК"
        self.btn_all = QPushButton("ОБЩИЙ СПИСОК")
        self.btn_all.setFont(QFont("Segoe UI", 10, QFont.Bold))
        self.btn_all.setCursor(Qt.PointingHandCursor)
        self.btn_all.setStyleSheet(f"background-color: {ACCENT_BLUE}; color: white; padding: 12px; border-radius: 5px;")

```

```

self.btn_all.clicked.connect(lambda: self.change_material("BCE ЗАМЕРЫ"))
left_layout.addWidget(self.btn_all)

self.buttons_layout = QVBoxLayout()
left_layout.addLayout(self.buttons_layout)
left_layout.addStretch()

# НОВАЯ ФИЧА: Кнопка ЭКСПОРТ ГРАФИКА
export_btn = QPushButton("📄 Экспорт Графика (PNG)")
export_btn.setFont(QFont("Segoe UI", 10, QFont.Bold))
export_btn.setCursor(Qt.PointingHandCursor)
export_btn.setStyleSheet("background-color: #8E44AD; color: white; padding: 10px; border-
radius: 5px;")
export_btn.clicked.connect(self.export_chart)
left_layout.addWidget(export_btn)

refresh_btn = QPushButton("🔄 ОБНОВИТЬ БАЗУ")
refresh_btn.setFont(QFont("Segoe UI", 11, QFont.Bold))
refresh_btn.setCursor(Qt.PointingHandCursor)
refresh_btn.setStyleSheet(
    "background-color: #2ea043; color: white; padding: 12px; border-radius: 5px; margin-top:
10px;")
refresh_btn.clicked.connect(self.refresh_data)
left_layout.addWidget(refresh_btn)

main_layout.addWidget(left_panel)

# --- ПРАВАЯ ПАНЕЛЬ ---
right_panel = QWidget()
right_layout = QVBoxLayout(right_panel)

# ВЕРХ: График
chart_frame = QFrame()
chart_frame.setStyleSheet(f"background-color: {BG_PANEL}; border-radius: 8px;")
chart_layout = QVBoxLayout(chart_frame)

plt.style.use('dark_background')
self.fig, self.ax = plt.subplots(figsize=(8, 3))
self.fig.patch.set_facecolor(BG_PANEL)
self.ax.set_facecolor(BG_PANEL)
self.canvas = FigureCanvas(self.fig)
chart_layout.addWidget(self.canvas)
right_layout.addWidget(chart_frame, stretch=4)

# СРЕДИНА: Карточка аналитики (УВЕЛИЧЕН ШРИФТ И ДОБАВЛЕН ФИЗИЧЕСКИЙ
ВЫВОД)
self.stat_frame = QFrame()
self.stat_frame.setStyleSheet(f"background-color: #2A2A2A; border-radius: 8px;")
stat_layout = QHBoxLayout(self.stat_frame)
self.stat_text = QLabel("Режим общего обзора. Выберите материал слева для глубокого
анализа.")
self.stat_text.setFont(QFont("Segoe UI", 12))

```



```

self.stat_text.setStyleSheet("color: #E0E0E0; padding: 10px;")
stat_layout.addWidget(self.stat_text)
right_layout.addWidget(self.stat_frame, stretch=1)

# НИЗ: Разделен пополам
bottom_layout = QHBoxLayout()

# Анимация
anim_frame = QFrame()
anim_frame.setStyleSheet(f"background-color: {BG_PANEL}; border-radius: 8px;")
anim_layout = QVBoxLayout(anim_frame)
anim_title = QLabel("Живая симуляция (Real-time)")
anim_title.setFont(QFont("Segoe UI", 11, QFont.Bold))
anim_layout.addWidget(anim_title, alignment=Qt.AlignCenter)

self.radar_anim = RadarAnimation()
anim_layout.addWidget(self.radar_anim)
bottom_layout.addWidget(anim_frame, stretch=1)

# Таблица (Все 6 колонок)
table_frame = QFrame()
table_frame.setStyleSheet(f"background-color: {BG_PANEL}; border-radius: 8px;")
table_layout = QVBoxLayout(table_frame)

self.table_title = QLabel("Журнал измерений (Все)")
self.table_title.setFont(QFont("Segoe UI", 11, QFont.Bold))
table_layout.addWidget(self.table_title, alignment=Qt.AlignCenter)

self.table = QTableWidgetItem(0, 6)
self.table.setHorizontalHeaderLabels(["№", "Материал", "е", "Толщина", "Дистанция",
"Энергия"])
self.table.verticalHeader().setVisible(False)
self.table.setEditTriggers(QAbstractItemView.NoEditTriggers)
self.table.setSelectionBehavior(QAbstractItemView.SelectRows)

header = self.table.horizontalHeader()
header.setSectionResizeMode(0, QHeaderView.ResizeToContents)
header.setSectionResizeMode(1, QHeaderView.Stretch)
header.setSectionResizeMode(2, QHeaderView.ResizeToContents)
header.setSectionResizeMode(3, QHeaderView.ResizeToContents)
header.setSectionResizeMode(4, QHeaderView.ResizeToContents)
header.setSectionResizeMode(5, QHeaderView.ResizeToContents)

self.table.setStyleSheet(f"""
    QTableWidgetItem {{ background-color: #161A1E; color: #D4D4D4; border: 1px solid #333;
gridline-color: #333; }}
    QHeaderView::section {{ background-color: #2D2D30; color: white; padding: 6px; font-
weight: bold; border: 1px solid #1e1e1e; }}
    QTableWidgetItem::item:selected {{ background-color: #007acc; }}
""")
table_layout.addWidget(self.table)
bottom_layout.addWidget(table_frame, stretch=1)

```

```

right_layout.addLayout(bottom_layout, stretch=4)
main_layout.addWidget(right_panel)

def change_material(self, mat):
    self.current_material = mat
    self.refresh_data()

def refresh_data(self):
    data_dict = self.engine.get_summary_data()
    raw_data = self.engine.get_raw_history(self.current_material)

    self.build_buttons(data_dict)
    self.update_table(raw_data)
    self.update_chart(data_dict)
    self.update_analytics()

    energy = data_dict.get(self.current_material, 100) if self.current_material != "BCE ЗАМЕРЫ"
else 100
    self.radar_anim.set_data(self.current_material, energy)

def build_buttons(self, data_dict):
    for i in reversed(range(self.buttons_layout.count())):
        self.buttons_layout.itemAt(i).widget().setParent(None)

    if not data_dict:
        return

    for mat in data_dict.keys():
        btn = QPushButton(mat.upper())
        btn.setFont(QFont("Segoe UI", 10))
        btn.setCursor(Qt.PointingHandCursor)
        if mat == self.current_material:
            btn.setStyleSheet("background-color: #e51400; color: white; padding: 10px; border-ra-
dus: 5px;")
        else:
            btn.setStyleSheet("background-color: #333337; color: white; padding: 10px; border-ra-
dus: 5px;")

        btn.clicked.connect(lambda checked, m=mat: self.change_material(m))
        self.buttons_layout.addWidget(btn)

def update_table(self, raw_data):
    self.table_title.setText(f"Журнал измерений ( {self.current_material} )")
    self.table.setRowCount(len(raw_data))
    for row_idx, row_data in enumerate(raw_data):
        for col_idx, value in enumerate(row_data[:6]):
            item = QTableWidgetItem(str(value))
            item.setTextAlignment(Qt.AlignCenter)
            self.table.setItem(row_idx, col_idx, item)

def update_chart(self, data_dict):

```

```

self.ax.clear()
if not data_dict: return

materials = list(data_dict.keys())
energies = list(data_dict.values())

colors = [ACCENT_RED if m == self.current_material else ACCENT_BLUE for m in materials]

bars = self.ax.bar(materials, energies, color=colors, width=0.4)

for bar in bars:
    yval = bar.get_height()
    self.ax.text(bar.get_x() + bar.get_width() / 2.0, yval + 2, f"{yval:.1f}",
                  ha='center', va='bottom', color='white', fontweight='bold', fontsize=11)

self.ax.set_ylim(0, 110)
self.ax.grid(axis='y', linestyle=':', alpha=0.2)
self.ax.spines['top'].set_visible(False)
self.ax.spines['right'].set_visible(False)
self.ax.spines['left'].set_color('#444')
self.ax.spines['bottom'].set_color('#444')
self.ax.tick_params(colors=TEXT_LIGHT)
self.ax.set_title(f"Спектр отражения 24 ГГц", color='white', fontsize=14, fontweight='bold',
pad=10)
self.fig.tight_layout()
self.canvas.draw()

def update_analytics(self):
    if self.current_material == "ВСЕ ЗАМЕРЫ":
        self.stat_text.setText(
            "Режим общего обзора. В таблице показаны все замеры.\nВыберите материал слева
для получения физических расчетов.")
        return

stats = self.engine.get_material_stats(self.current_material)
if stats:
    loss = 100 - stats['avg_energy']

    # ИИ-советник (Физический вывод)
    if stats['avg_energy'] > 85:
        conclusion = "Материал является РАДИОПРОЗРАЧНЫМ (СВЧ-линза)."
    elif stats['avg_energy'] > 30:
        conclusion = "Материал обладает СРЕДНИМ ПОГЛОЩЕНИЕМ СВЧ-волн."
    else:
        conclusion = "ПОЛНАЯ РАДИОБЛОКИРОВКА. Сильное затухание сигнала (СВЧ-
щит)."
```

```

text = (f" АНАЛИТИКА МАТЕРИАЛА: {self.current_material.upper()} \n"
        f"Диэлектрическая проницаемость (ε): {stats['epsilon']} | Толщина преграды:
{stats['thickness']} мм | Затухание СВЧ-сигнала: {loss:.1f}%\n"
        f"Физический вывод: {conclusion}")
self.stat_text.setText(text)

```

```

def export_chart(self):
    """НОВАЯ ФИЧА: Сохранение графика в PNG для курсовой"""
    try:
        filename = f"Graph_{self.current_material}.png"
        self.fig.savefig(filename, dpi=300, bbox_inches='tight', facecolor=BG_PANEL)
        QMessageBox.information(self, "Успех!",
                                f"График сохранен в высоком качестве (300 DPI)\nв папку проекта
как:\n{filename}")
    except Exception as e:
        QMessageBox.critical(self, "Ошибка", f"Не удалось сохранить график: {e}")

```

```

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = RadarApp()
    window.show()
    sys.exit(app.exec_())

```

data_engine:

```

import pandas as pd
import os

```

```

class DataEngine:
    def __init__(self, filename='radar_physics_data.csv'):
        self.filename = filename

    def get_summary_data(self):
        """Читает CSV и возвращает усредненные данные для графика"""
        if not os.path.exists(self.filename):
            return {}

        try:
            df = pd.read_csv(self.filename, sep=';', encoding='utf-8-sig')
            df['Материал'] = df['Материал'].astype(str).str.strip().str.capitalize()

            summary = df.groupby('Материал')['Энергия'].mean().reset_index()
            summary = summary.sort_values(by='Энергия', ascending=False)

            return dict(zip(summary['Материал'], summary['Энергия']))
        except Exception as e:
            print(f"Ошибка в data_engine (summary): {e}")
            return {}

    def get_raw_history(self, material_filter="ВСЕ ЗАМЕРЫ"):
        """Возвращает данные для таблицы (все 6 колонок) с фильтрацией"""
        if not os.path.exists(self.filename):
            return []

        try:
            df = pd.read_csv(self.filename, sep=';', encoding='utf-8-sig')
            df['Материал'] = df['Материал'].astype(str).str.strip().str.capitalize()

```

```

# Фильтруем, если выбран конкретный материал
if material_filter != "ВСЕ ЗАМЕРЫ":
    df = df[df['Материал'] == material_filter]

# Сортируем по номеру эксперимента (новые сверху)
df = df.sort_values(by='Эксперимент', ascending=False)

return df.values.tolist()
except Exception as e:
    print(f"Ошибка в data_engine (history): {e}")
    return []

def get_material_stats(self, material):
    """Считает глубокую аналитику для выбранного материала"""
    if not os.path.exists(self.filename) or material == "ВСЕ ЗАМЕРЫ":
        return None

    try:
        df = pd.read_csv(self.filename, sep=';', encoding='utf-8-sig')
        df['Материал'] = df['Материал'].astype(str).str.strip().str.capitalize()
        df_mat = df[df['Материал'] == material]

        if df_mat.empty: return None

        stats = {
            "count": len(df_mat),
            "avg_dist": round(df_mat['Дистанция(см)'].mean(), 1),
            "avg_energy": round(df_mat['Энергия'].mean(), 1),
            "epsilon": df_mat['ε Проницаемость'].iloc[0],
            "thickness": df_mat['Толщина(мм)'].iloc[0]
        }
        return stats
    except:
        return None

```

radar_canvas:

```

from PyQt5.QtWidgets import QWidget
from PyQt5.QtCore import QTimer, Qt, QRectF
from PyQt5.QtGui import QPainter, QColor, QPen, QBrush, QFont

```

```

class RadarAnimation(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setMinimumHeight(250)
        self.material = "ВСЕ ЗАМЕРЫ"
        self.energy = 100.0
        self.phase = 0
        self.timer = QTimer(self)
        self.timer.timeout.connect(self.update_animation)
        self.timer.start(30)

```

```

def set_data(self, material, energy):
    self.material = material
    self.energy = energy

def update_animation(self):
    self.phase += 6
    if self.phase > 100:
        self.phase = 0
    self.update()

def paintEvent(self, event):
    painter = QPainter(self)
    painter.setRenderHint(QPainter.Antialiasing)

    width, height = self.width(), self.height()
    mid_y = height // 2
    radar_x, obstacle_x, foil_x = 50, width // 2, width - 80

    # Фон
    painter.fillRect(0, 0, width, height, QColor("#161A1E"))

    # Радар
    painter.setBrush(QBrush(QColor("#E51400")))
    painter.setPen(Qt.NoPen)
    painter.drawRect(radar_x - 20, mid_y - 20, 40, 40)
    painter.setBrush(QBrush(QColor("#FFD700")))
    painter.drawRect(radar_x, mid_y - 10, 10, 20)
    painter.setPen(QColor("#FFFFFF"))
    painter.setFont(QFont("Arial", 9, QFont.Bold))
    painter.drawText(radar_x - 22, mid_y + 35, "РАДАР")

    if self.material == "BCE ЗАМЕРЫ":
        # Режим сканирования (просто радар и длинные волны)
        self.draw_waves(painter, radar_x, width, width, mid_y, True)
        painter.setPen(QColor("#888888"))
        painter.drawText(width // 2 - 50, mid_y, "РЕЖИМ ОБЗОРА")
        return

    # Фольга
    painter.setPen(QPen(QColor("#CCCCCC"), 4))
    painter.drawLine(foil_x, mid_y - 60, foil_x, mid_y + 60)
    painter.setPen(QColor("#FFFFFF"))
    painter.drawText(foil_x - 25, mid_y - 70, "МИШЕНЬ")

    # Преграда и Волны
    self.draw_obstacle(painter, obstacle_x, mid_y)
    self.draw_waves(painter, radar_x, obstacle_x, foil_x, mid_y, False)

def draw_obstacle(self, painter, x, y):
    mat_lower = self.material.lower()
    if "воздух" in mat_lower: return

```

```

if "пластик" in mat_lower:
    painter.setBrush(QBrush(QColor(100, 200, 255, 60)))
    painter.setPen(QPen(QColor(100, 200, 255, 150), 2))
elif "вод" in mat_lower:
    painter.setBrush(QBrush(QColor(10, 100, 200, 200)))
    painter.setPen(QPen(QColor(50, 150, 255), 2))
else:
    painter.setBrush(QBrush(QColor(150, 150, 150, 100)))
    painter.setPen(QPen(QColor(200, 200, 200), 2))

painter.drawRoundedRect(x - 20, y - 70, 40, 140, 10, 10)
painter.setPen(QColor("#FFFFFF"))
painter.drawText(x - 30, y - 80, self.material)

def draw_waves(self, painter, start_x, mid_x, end_x, y, is_all_mode):
    max_radius = end_x - start_x if (self.energy > 30 or is_all_mode) else mid_x - start_x

    for i in range(4):
        radius = ((self.phase + i * 25) % 100) / 100.0 * max_radius
        if radius <= 0: continue

        alpha = int(255 * (1 - (radius / max_radius)))

        if is_all_mode:
            color = QColor(100, 150, 255, alpha)
        elif self.energy > 30:
            color = QColor(0, 255, 150, alpha)
        else:
            color = QColor(255, 50, 50, alpha)

        painter.setPen(QPen(color, 3))
        rect = QRectF(start_x - radius, y - radius, radius * 2, radius * 2)
        painter.drawArc(rect, -30 * 16, 60 * 16)

    if self.energy <= 30 and not is_all_mode:
        painter.setPen(QPen(QColor(255, 0, 0, 150), 4, Qt.DotLine))
        painter.drawLine(mid_x - 25, y - 50, mid_x - 25, y + 50)

```

ПРИЛОЖЕНИЕ В

(обязательное)

Руководство системного программиста

Аппаратно-программный комплекс реализован на двух уровнях: микропрограммном (язык C++, платформа *Arduino IDE*) и прикладном (язык *Python 3*). Программа предназначена для высокоскоростного сбора, пространственной фильтрации и визуализации телеметрических данных, получаемых от СВЧ-радары *HLK-LD2410C*. Для работы программной части требуется наличие установленной среды выполнения *Python* версии 3.8 или выше, а также библиотек *PyQt5*, *Pandas*, *Matplotlib* и *PySerial*. В качестве хранилища данных используется локальная плоская база данных в формате *CSV*, взаимодействие с которой осуществляется средствами файловой системы операционной системы без разворачивания выделенного сервера СУБД.

Для начала работы система должна установить соединение с аппаратным узлом. Конфигурация *COM*-порта и скорости обмена данными (115200 бод) задается в теле скрипта. После запуска приложение предоставляет интерактивный графический интерфейс. Микроконтроллер осуществляет непрерывный опрос датчика по аппаратному интерфейсу *UART2* на скорости 256000 бод, выполняет первичную пространственную фильтрацию (отсечение целей ближе 75 см) и передает валидные пакеты на персональный компьютер. Прикладное программное обеспечение асинхронно считывает данные из последовательного порта, агрегирует их с помощью структур *DataFrame* и записывает усредненные результаты в *CSV*-файл для защиты файловой системы от избыточного количества транзакций.

В ходе работы комплекса могут возникнуть системные сложности. Например, программа сообщит об ошибке «*SerialException*», если указанный *COM*-порт недоступен, занят другим приложением (например, монитором порта в *Arduino IDE*) или микроконтроллер не подключен. Также может возникнуть ошибка доступа «*PermissionError*», если файл локальной базы данных открыт в стороннем табличном редакторе в момент записи телеметрии. При возникновении ошибок рекомендуется проверить корректность конфигурационных констант последовательного порта, убедиться в целостности *USB*-подключения и проверить наличие питания 5В на аппаратном полигоне.

ПРИЛОЖЕНИЕ Г

(обязательное)

Руководство пользователя

Десктопное приложение для анализа СВЧ-телеметрии позволяет в реальном времени получать радиолокационные данные, сохранять результаты экспериментов в базу данных и визуализировать эффекты прохождения электромагнитных волн сквозь различные диэлектрические преграды. Разработанное с использованием фреймворка *PyQt5* и математических библиотек *Pandas* и *Matplotlib*, приложение идеально подходит для проведения лабораторных работ по радиофизике. Для работы требуется операционная система *Windows* или *Linux*, установленный интерпретатор *Python* и физическое подключение измерительного стенда по *USB*.

Чтобы начать работу, необходимо запустить скрипт приложения. На экране появится графический интерфейс, функционально разделенный на три зоны: панель управления и навигации (слева), зону аналитики и графиков (сверху справа) и зону мониторинга с таблицей сырых данных и анимацией физического процесса (снизу справа). Пользователю предоставляется возможность выбрать ранее протестированный материал из списка для просмотра исторических данных или инициировать новый эксперимент.

Для проведения нового измерения необходимо зафиксировать эталонную металлическую мишень на дистанции 100 см от радара, а исследуемую преграду (воздух, пластик, вода) разместить на отметке 50 см. В консоли приложения требуется ввести название материала, его справочную диэлектрическую проницаемость и толщину. Запуск аппаратного триггера осуществляется бесконтактно: оператору необходимо совершить движение рукой в ближней зоне радара (до 75 см). Система активирует визуальную индикацию, предоставит задержку в 3 секунды для эвакуации оператора из рабочей зоны и произведет замер.

После успешного сбора телеметрии программа автоматически вычислит среднюю дистанцию и уровень энергии, обновит базу данных и перестроит графики. Пользователю необходимо обратить внимание на аналитическое окно «ИИ-советник», которое автоматически классифицирует преграду как радиопрозрачную среду, материал со средним поглощением или СВЧ-щит. Для сохранения результатов эксперимента предусмотрена кнопка экспорта гистограммы в растровый формат *PNG* высокого разрешения.

ПРИЛОЖЕНИЕ Д

(обязательное)

Руководство программиста

Программное обеспечение измерительного комплекса представляет собой распределенную систему, разработанную с использованием языков C++ (микропрограммный уровень) и *Python* (прикладной уровень). Решение спроектировано для жесткого реал-тайм сбора данных и их последующей математической обработки.

Микропрограммный код для платформы *ESP32* реализован в среде *Arduino IDE*. Логика работы построена на концепции конечного автомата с применением неблокирующих задержек на базе функции *millis()*. Это позволило организовать параллельную фоновую очистку приемного *UART*-буфера радара, исключив переполнение оперативной памяти и потерю пакетов телеметрии. Для управления адресной светодиодной индикацией *WS2812B* интегрирована оптимизированная библиотека *Adafruit_NeoPixel*.

Архитектура десктопного приложения базируется на паттерне *Model-View-Controller (MVC)* и разделена на изолированные модули: *data_engine.py* (логика работы с *CSV*-файлами и агрегация данных через *Pandas DataFrame*), *radar_canvas.py* (графический движок на базе *QPainter* для симуляции волнового фронта) и *main_app.py* (контроллер графического интерфейса *PyQt5*). Математическая визуализация спектра поглощения интегрирована нативно через класс *FigureCanvasQTAgg* библиотеки *Matplotlib*.

Тестирование математического ядра и подсистемы хранения данных инкапсулировано в отдельном скрипте с использованием библиотеки *unittest*. В целях предотвращения искажения реальных экспериментальных данных алгоритм тестирования применяет *Mock*-объекты (временные файлы-заглушки). Тесты верифицируют корректность обработки исключений, точность вычисления среднего арифметического значения СВЧ-энергии и алгоритмы нормализации строковых входных данных.

Программный комплекс имеет слабо связанную архитектуру. Благодаря использованию библиотеки *Pandas* логика агрегации телеметрии легко масштабируется, а модульность приложения позволяет в будущем интегрировать протоколы интернета вещей (*MQTT/WebSocket*) без внесения изменений в низкоуровневые механизмы работы с измерительным оборудованием.



Отчет о проверке

Автор: slavaa11@gmail.com / ID: 12144831

Проверяющий: slavaa11@gmail.com / ID: 12144831

Название документа: курсоваясеноженский.docx

РЕЗУЛЬТАТЫ ПРОВЕРКИ

Совпадения:
Не менее 5%



Оригинальность:
Не более 86%



Цитирования:
Не менее 2%



Самοцитирования:
Не менее 7%



«Совпадения», «Цитирования», «Самοцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.

- **Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самοцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- **Самοцитирования** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самοцитирования» — это доля фрагментов текста, отнесенных к самοцитированию, в общем объеме текста.
- **Цитирования** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы, библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- **Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- **Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- **Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 1

Тип документа: *.docx

Дата проверки: 17.05.2026 09:58:17

Дата корректировки: Нет

Количество страниц: 50

Символов в тексте: 297653

Слов в тексте: 10890

Число предложений: 376

Комментарий: Сеноженский ИТП-31